

DATA420-25S1 (C)

Assignment 1

GHCN Data Analysis using Spark

Ying Yan

99266985

[CONTENT](#)

Background	3
Processing Q1	
(a)	3
(b)	3
(c)	4
Processing Q2	
(a)	4
(b)	4
(c)	5
(d)	6
Processing Q3	
(a)	6
(b)	6
(c)	7
(d)	7
(e)	8
Processing Q4	
(a)	9
(b)	9
Analysis Q1	
(a)	9
(b)	9
(c)	10
Analysis Q2	
(a)	10
(b)	10
Analysis Q3	
(a)	10
(b)	10
(c)	11
Visualizations Q1	
(a)	11
(b)	12
Visualizations Q2	
(a)	15
(b)	18
Conclusion	19
Appendices	19
Reference	39

Background

This assignment focuses on analysing weather data from the Global Historical Climatology Network (GHCN), a comprehensive database of climate summaries collected from over 100,000 land surface stations in 200 countries and territories around the world. The data covers over 250 years and is collected from more than 20 independent sources.

Processing Q1

(a) Data structure

Following are the data structure and the data formats of how the weather data in the GHCN dataset is stored in HDFS:

```

/
└─/ghcnd
    └─/ghcnd/ghcnd-countries.txt
    └─/ghcnd/daily
        └─/ghcnd/daily/1750.csv.gz
        └─/ghcnd/daily/1763.csv.gz
        └─.....
        └─/ghcnd/daily/2024.csv.gz
        └─/ghcnd/daily/2025.csv.gz
    └─/ghcnd/ghcnd-inventory.txt
    └─/ghcnd/ghcnd-states.txt
    └─/ghcnd/ghcnd-stations.txt
  
```

There are four text formatted files, which are “countries”, “inventory”, “states”, and “stations”, and one directory, which is “daily”. “daily” contains all the csv.gz formatted files and one file per year from the year 1750 to the year 2025.

(b) Visualising size change in daily data

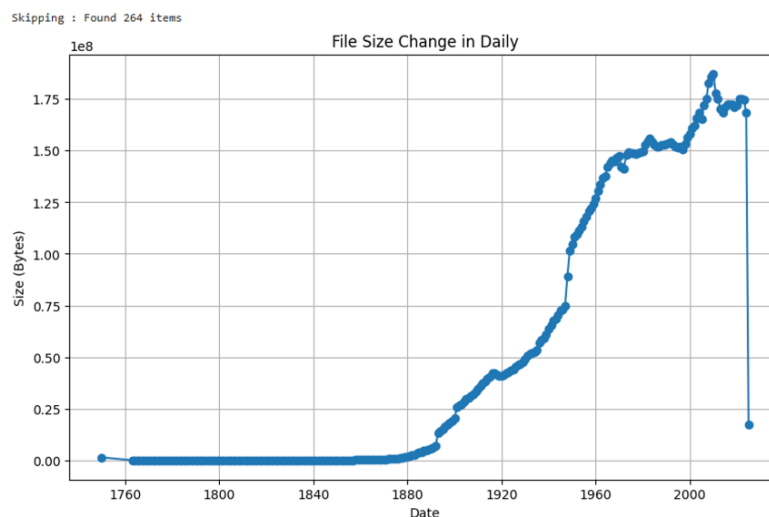


Figure 1 File size change in daily

There are 264 years recorded in daily. From Figure 1, the file size remained relatively small and constant before the year 1880. The size then increased sharply in the following years and peaked around the year 2010 at 178.6 MB. The file size reduced dramatically due to partial penetration into the current year.

(c) Total data size

Size	File Name
3.6 KB	ghcnd-countries.txt
33.6 MB	ghcnd-inventory.txt
1.1 KB	ghcnd-states.txt
10.6 MB	ghcnd-stations.txt
13.0 GB	daily
13.1 GB	Total

Table 1 File size

The total size of all the data is 13.1 GB. According to Table 1, 3.6 KB is countries, 33.6 MB is inventory, 1.1 KB is states, 10.6 MB is stations, and 13.0 GB is daily.

Processing Q2

(a) Define a schema for the daily dataset

For “daily”, the description of the data and in the assignment is accurate, apart from the DATE column, which should be in date data type and the “OBSERVATION TIME” column should be timestamp data type, but they are formatted as string data type, then it has been designed to load the schema into spark manually as following for the 8 columns:

1. “ID”, string data type,
2. “DATE”, string data type,
3. “ELEMENT”, string data type,
4. “VALUE”, float data type,
5. “MEASUREMENT FLAG”, string data type,
6. “QUALITY FLAG”, string data type,
7. “SOURCE FLAG”, string data type,
8. “OBSERVATION TIME”, string data type

(b) Load daily data with defined schema

Based on the description of the daily data, I initially defined the “DATE” field as “DateType()”, since it is a date formatted as “YYYYMMDD”. However, this caused issues during loading because Spark’s “DateType()” expects values in the format “yyyy-MM-dd”. Therefore, I changed the DATE field to “StringType()” followed by converting it to date format manually to load the data correctly. The remaining fields, “ID”, “ELEMENT”, “MEASUREMENT FLAG”, “QUALITY FLAG”, and “SOURCE FLAG” are all described as “Character”, so I defined these columns as “StringType()”, the “VALUE” as “FloatType()” as describes as “Real”. With the “OBSERVATION TIME” although it

is described as “Time” type, It is formatted as “HHMM”, so I used “StringType()” to match the raw data format.

(c) Load metadata tables and extract columns

For “stations”, it should have 9 columns, the schema could not be applied when we tried to load them into Spark, we need to fix the width text format, which requires columns to be extracted by selecting the relevant substring of the text. This will be achieved by loading it into a dataframe with one text column and using a select statement to extract substring, trim any whitespace, and apply the datatype from the schema based on actual column data type as following:

1. Column name “ID”, from the first to 11th character of each row of its value, which length is 11 characters and it is a string data type;
2. Column name “LATITUDE”, from the 13th to 20th character of each row of its value, which length is 8 characters and it is a float data type;
3. Column name “LONGITUDE”, from the 22th to 30th character of each row of its value, which length is 9 characters and it is a float data type;
4. Column name “ELEVATION”, from the 32th to 37th character of each row of its value, which length is 6 characters and it is a float data type;
5. Column name “STATE”, from the 39th to 40th character of each row of its value, which length is 2 characters and it is a string data type;
6. Column name “NAME”, from the 42th to 71th character of each row of its value, which length is 30 characters and it is a string data type;
7. Column name “GSN FLAG”, from the 73th to 75th character of each row of its value, which length is 3 characters and it is a string data type;
8. Column name “HCN/CRN FLAG”, from the 77th to 79th character of each row of its value, which length is 3 characters and it is a string data type;
9. Column name “WMO ID”, from the 81th to 85th character of each row of its value, which length is 5 characters and it is a string data type;

For “countries”, it should have 2 columns to read into a dataframe as following:

1. Column name “CODE”, from the first to the second character of each row of its value, which length is 2 characters, and it is a string data type.
2. Column name “NAME”, from the 4th to 64th character of each row of its value, which length is 61 characters, and it is a string data type.

For “states”, it should have 2 columns, we need to fix the width of each column since it is a text formatted file:

1. Column name “CODE”, from the first to the second character of each row of its value, which length is 2 characters, and it is a string data type.
2. Column name “NAME”, from the 4th to 50th character of each row of its value, which length is 47 characters, and it is a string data type.

For “inventory”, it should have 6 columns to load into a dataframe as following:

1. Column name “ID”, from the first to 11th character of each row of its value, which length is 11 characters, and it is a string data type;
2. Column name “LATITUDE”, from the 13th to 20th character of each row of its value, which length is 8 characters, and it is a float data type;
3. Column name “LONGITUDE”, from the 22th to 30th character of each row of its value, which length is 9 characters, and it is a float data type;
4. Column name “ELEMENT”, from the 32th to 35th character of each row of its value, which length is 4 characters, and it is a string data type;
5. Column name “FIRSTYEAR”, from the 37th to 40th character of each row of its value, which length is 4 characters, and it is a string data type;
6. Column name “LASTYEAR”, from the 42th to 45th character of each row of its value, which length is 4 characters, and it is a string data type.

(d) Counting rows in metadata tables

Data Table	Number of Rows
stations	129657
states	74
countries	219
inventory	765615

Table 2 Row number in each metadata table

According to Table 2 above, there are 129657 rows in “stations”, and each station ID is unique, so there are 129657 stations;

There are 74 rows in “states”, and each state CODE is unique, so there are 74 states in the data;

There are 219 rows in “countries”, and each country CODE is unique, so there are 219 countries in the data;

There are 765615 rows in “inventory”.

Processing Q3

(a) Extracting country code from station IDs

Based on the first two characters of the stations dataframe denoting the Federal Information Processing Standards (FIPS) country code, stated in the readme file, a new column, “COUNTRY CODE”, has been created by substringing them.

(b) LEFT JOIN stations with countries

Once the country code information is available in the stations dataframe, then we can join the countries dataframe on country code to fetch the country name information. Stations dataframe left join has been conducted to join countries dataframe. The “CODE” and “NAME” have been renamed to “COUNTRY CODE” and “COUNTRY NAME”, respectively, to match the joining column

name, and to clarify that the “NAME” refers to the country name and avoid column name redundancy.

(c) ILEFT JOIN stations and states

Next, we can fetch the state information for the stations with the United States region by joining with the states dataframe on “STATE”. By using the same method, use the output dataframe from step (b), left join states dataframe with the “CODE” and “NAME” renamed as “STATE” and “STATE NAME” to facilitate the join on “STATE” and to clarify the state name feature and avoid column name redundancy.

(d) Analyzing station activity from inventory

The inventory dataframe has been grouped by “ID” and then aggregated the minimum and maximum values of the first active year and last active year across all elements for each station. For example, the first year station ID USC00016329 was active in 1960, and the last active year was in 1976.

To determine the number of unique elements collected by each station, the inventory dataframe was grouped by “ID” and then aggregated using the collect_set() Spark function, combined with the array_sort() function, on the “ELEMENT” feature to obtain a distinct, ordered set of elements for each station. This information was stored in a new column called “DISTINCT ELEMENT.” Additionally, another column, “DISTINCT ELEMENT COUNT,” was created to count the number of elements in each set using the size() function. The results show that station AE000041196 has recorded four distinct elements: precipitation, average daily temperature, maximum temperature, and minimum temperature.

To count the distinct core and other elements separately, a core element list has been created to define the names of core elements. After that, the “DISTINCT ELEMENT” information can be reused to check whether the element in each set has been included in the core element list, store them in a new set, respectively, and then count the size of each set to derive the element count. Two Spark user-defined functions have been created to facilitate the core and other element checking and new sets forming. New columns “CORE ELEMENT COUNT” and “OTHER ELEMENT COUNT” have been created to call the user-defined functions and store and counted information. In particular, of the four elements collected by station AE000041196, three of them are core elements, and one is the other element.

To count how many stations collected all five core elements, it can be achieved by simply counting how many rows in the aggregated inventories dataframe with “CORE ELEMENT COUNT” equal to five. The result is 20504.

Finally, the where clause has been conducted to filter the rows with the “DISTINCT ELEMENT” column of the dataframe contains the keyword, “PRCP”, by using the Spark function, “array_contains()”, and then the additional condition that the number of a distinct element is

one (“DISTINCT ELEMENT COUNT” equals to one), meaning that the station one collected one distinct element, and the element is precipitation. The result is 16313.

(e) [Enrich stations table and save the output](#)

The enriched station dataframe with country name and state information has been left joined with aggregated inventory dataframe that includes the first and last year a given station was active on “ID”, and then left joined with the aggregated inventory dataframe output from question d), contains distinct element count information. Please see the following table 3 for details:

Column Name	Data Type	Description
ID	String	Unique station identifier
STATE	String	Two character US state code
COUNTRY CODE	String	Two character FIPS country code
LATITUDE	Float	Latitude of the station (in decimal degrees)
ELEVATION	Float	The elevation of the station (in meters, missing = -999.9)
NAME	String	The name of the station
GSN FLAG	String	A flag that indicates whether the station is part of the GCOS Surface Network (GSN)
HCN/CRN FLAG	String	A flag that indicates whether the station is part of the U.S. Historical Climatology Network (HCN) or U.S. Climate Reference Network (CRN)
WMO ID	String	The World Meteorological Organization (WMO) number for the station
COUNTRY NAME	String	The name of the country
STATE NAME	String	The name of the state, territory or province
ACTIVE FIRSTYEAR	Integer	The first year of each station actively collected any element
ACTIVE LASTYEAR	Integer	The last year of each station actively collected any element
DISTINCT ELEMENT	Array(String)	The distinct elements collected by each station
DISTINCT ELEMENT COUNT	Integer	The number of the distinct elements collected by each station
CORE ELEMENT COUNT	Integer	The number of the core elements collected by each station
OTHER ELEMENT COUNT	Integer	The number of the other elements collected by each station

[Table 3 Enriched stations table](#)

Processing Q4

(a) Check missing stations via LEFT JOIN

During left join, Spark needs to shuffle or broadcast data across nodes to align matching keys between the two dataframes. It could be computationally expensive for large datasets. Given daily has 13.0 GB of data, a left join between daily and stations would be inefficient due to the large size of the daily dataset.

A more efficient way is that we can use LEFT ANTI JOIN. This join type returns only the rows from the left dataframe daily and do not need to match in the stations dataframe, it reduces memory and shuffle overhead. It only validates whether the existence of the key in the right dataframe is also in the left dataframe.

(b) Count missing stations in daily data

All stations can be found in the daily file for 2025.

Analysis Q1

(a) Counting Total and Active Stations

To count the total number of stations dataframe, use “distinct()” on select stations column ID. I filtered any “ACTIVE LAST YEAR” equal to 2025 to count the stations that are active in 2025. There are 129657 stations in total, and 30735 stations are active so far in 2025.

Filter out any “GSN FLAG” equal to “GSN”, “HCN/CRN FLAG” equals to HCN and CRN accordingly first, then count:

There are 991 stations in GCOS Surface Network, there are 1218 stations in US Historical Climatology Network, there are 234 stations in US Climate Reference Network.

I filter any “GSN FLAG” value that is not empty, which is equal to zero, and any “HCN/CRN FLAG” is not empty at the same time, which means the record collected both records, then I count distinct. There are 15 stations in more than one of those networks.

(b) Counting Stations in the Southern Hemisphere

To count the number of stations in the Southern Hemisphere, to represent GPS coordinates with sub-meter precision, I convert the “LATITUDE” column’s data type to double type. Then count the stations with negative latitude. There are 25357 stations in the Southern Hemisphere.

Filter the “COUNTRY NAME” containing “United States” also not equal to the “United States” to get the result of a number of the stations in the territories of the United States but not in itself. There are 414 stations.

(c) Counting stations by country and state, and joining to Metadata

To count the total number of stations in each country, select and then group by the “COUNTRY CODE” column, followed by counting rows based on one row representing one distinct station. Once the intermediate dataframe is derived, it can be joined with the countries dataframe to incorporate the number of stations information into the countries dataframe. Finally, the enriched countries dataframe can be saved back into HDFS.

To count the total number of stations in each state, the same method was used except for the rows with null values excluded, given the states are the United States only. The enriched states dataframe was also saved into HDFS.

Analysis Q2

(a) Writing Geographical Distance Function

To create a User Define Function (UDF) to calculate the geographic distance between stations, the Haversine formula (Sinnott, 1984), which accounts for the Earth’s spherical shape, has been used to calculate the distance based on the latitude and longitude of stations. Then, converted to UDF function, also get a small subset of the dataframe by the limit of 100 rows, converted the data type for latitude and longitude to double type for a certain level of precision. The subset stations dataframe then made a cross-join to itself and removed repeated joined station pairs by using the array sort function and avoiding the same station pair to apply the function for testing purposes.

(b) Applying Distance Function to New Zealand (NZ) Stations

First, all NZ stations were filtered out by checking whether the country code was “NZ”. Then the same method was used for preparing the stations dataframe to apply the function. Finally, the result was sorted by distance. It appears that the “WELLINGTON AERO AWS” station (station ID NZM00093439) and the “PARAPARAUMU AWS” station (station ID NZ000093417) are the geographically closest stations in NZ at 50.53 km apart.

Analysis Q3

(a) Counting Rows in Daily Data

Loaded all the daily files into one schema and count the number of rows. There are 3139143397 rows in all daily files.

(b) Filtering Observations for Core Elements

To count the number of observations for each core element, first, define a list of core weather elements, including maximum temperature(TMAX), minimum temperature (TMIN), precipitation (PRCP), snowfall (SNOW), and snow depth (SNMD). Then, filter all “ELEMENT” match the five

core element strings, and count on the core elements. Cashed the counting result, and sorted the core element count dataframe descending by the observation result.

- Element "PRCP" has 107976077 observations in daily.
- Element "TMAX" has 460114659 observations in daily.
- Element "TMIN" has 458928768 observations in daily.
- Element "SNOW" has 359249644 observations in daily.
- Element "SNOD" has 300711620 observations in daily.
- Element "PRPC" has the most observations.

(c) Analyzing TMAX and TMIN Observations

To determine the number of observations where "TMAX" is recorded without a corresponding "TMIN", start by using the "isin()" function to filter the dataset for observations with elements "TMAX" and "TMIN". Next, group the filtered data by station "ID" and "DATE", then count the number of observations within each group and identify the first element in the group.

Focus on the groups where the count of observations is one, as these indicate cases where either "TMAX" or "TMIN" exists without a pair. Specifically, target the groups where the count is one and the element is "TMAX", since these represent "TMAX" records without corresponding "TMIN" entries. Groups with a count of two (containing both "TMAX" and "TMIN") are not relevant and should be excluded.

Finally, count the total number of these "TMAX-only" occurrences, and determine how many distinct stations are involved to find the number of unique stations contributing to these observations.

There are 10660214 observations of "TMAX" that do not have a corresponding observation of "TMIN", and there are 28754 unique stations contributed to these observations.

Visualizations Q1

(a) Filter and Save TMIN and TMAX Observations for NZ Stations

To obtain all daily observations with the elements "TMAX" and "TMIN" for NZ stations only, filter the dataframe using the isin function to match the "ELEMENT" column with the values "TMAX" and "TMIN." Additionally, substring the first two characters of the "ID" column to match "NZ," which represents the country code for New Zealand.

To determine how many years are covered by the observations, use the select method with the F.year() function to extract the year from the "DATE" column, then count the number of distinct years.

In total, there are 491,441 observations of the “TMAX” and “TMIN” elements collected by NZ stations, covering a span of 86 years.

(b) Plot Time Series for TMIN and TMAX Together For Each Station in NZ and The Entire Country.

To visualize the time series of “TMIN” and “TMAX” for each station in New Zealand, start by loading the stations dataset. Perform a left join between the daily dataframe and the stations dataframe to incorporate station names. Next, convert the Spark dataframe to a Pandas dataframe to facilitate plotting with Matplotlib. It is important to ensure that the “DATE” column retains its date data type during the conversion, as this is crucial for accurate analysis and visualization in the subsequent steps.

A for loop was used to plot the data for each station in New Zealand. Within the loop, the data was filtered for each specific station and then split into two separate datasets representing maximum and minimum temperatures. Daily time series plots were generated for each station, with the station name and ID included in the plot titles. Both the maximum and minimum temperatures for each station were displayed on the same plot, using different colors and corresponding legends. However, the data gaps were connected by straight lines, and the date ranges on the X-axis varied across different stations.

To show the data gaps, reindexing has been implemented by replacing the existing index of the dataframe with the “DATE” column as a datetime index. The dataframe is then reindexed using a date range series that spans from the first to the last date in the entire dataset, with a daily frequency. As a result, the rows corresponding to dates with missing data will have “None” values, which will appear as gaps in the plots.

To address issues with visualizing data over different date ranges, the limits of the x-axis have been set to one year before the first date and one year after the last date in the entire dataset. This ensures a consistent visualization across all plots.

Weekly (Figure 3), monthly (Figure 4), and yearly time series have been plotted by grouping the data by the “DATE” column with the specified frequency and aggregating temperature values by calculating the average. The data gaps are also reflected in the plots, using the same reindexing method described earlier with the specified frequency.

Finally, the time series plot for each station or for New Zealand at multiple date frequencies has been integrated into a function with three parameters: dataframe, date frequency, and station or entire NZ granularity (Figure 5). Within the function, two dictionaries are created to map to the corresponding frequency and granularity. These dictionaries determine whether to filter the dataframe for a specific station or for all stations in New Zealand, then group and aggregate the data accordingly. Each generated plot is saved into its corresponding folder, defined by date frequency, with the station ID or “NZ” combined with the frequency as “png” files.

Please refer to the following figures as examples of the time series visualisations, and the rest can be found in the appendices:

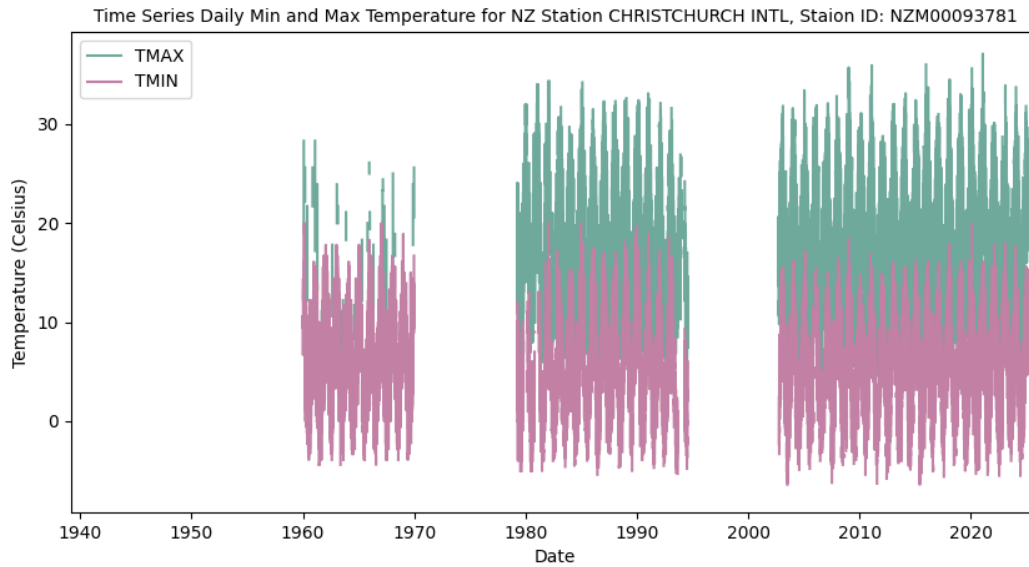


Figure 2 NZ station daily minimum and maximum temperature over time

Figure 2 shows that the Christchurch International Station has been collecting temperature readings since around 1960. However, there are data gaps between 1970 and 1978, as well as between 1995 and 2022. The average daily maximum temperature over time is around 18 degrees, while the average minimum temperature remains slightly below 10 degrees.

In Figure 3, the data for the weekly average temperature appears sparser. The New Plymouth AWS Station began collecting data around 1943, and it seems that data collection has continued up to the present year.

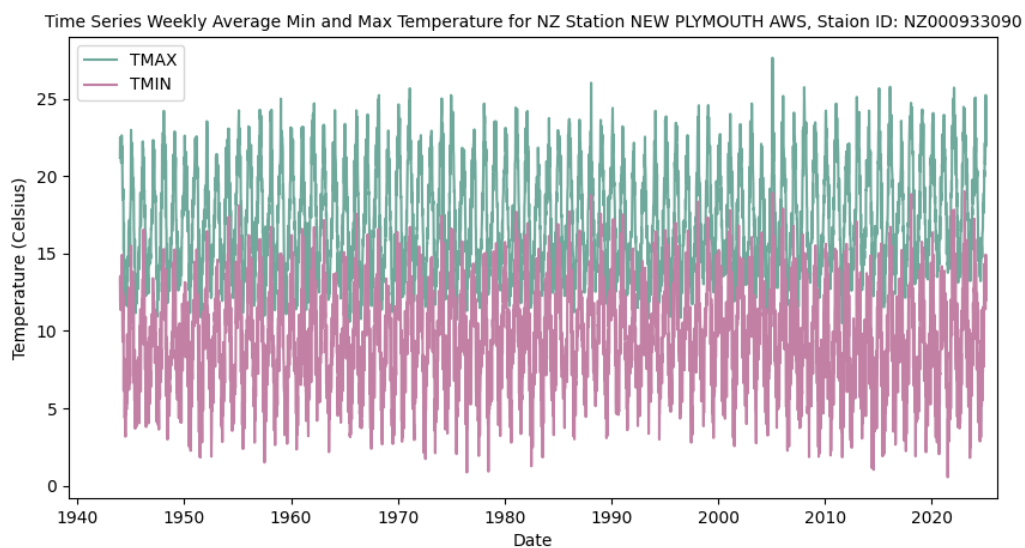


Figure 3 NZ station weekly average minimum and maximum temperature over time

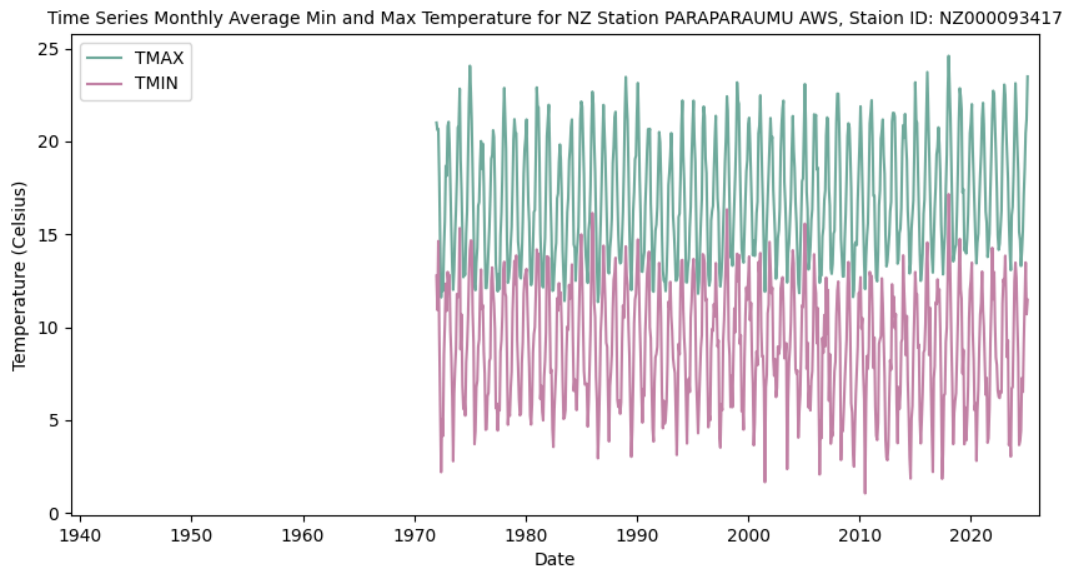


Figure 4 NZ station monthly average minimum and maximum temperature over time

Figure 4 shows the monthly average temperature recorded by the Paraparaumu AWS Station, with data collection starting in the early 1970s and continuing to the present. The average maximum and minimum temperatures are similar to those recorded at the Christchurch station over time.

Figure 5 presents the average temperature trends for the entire country. There was a decrease in temperature in the early 1940s, but both the maximum and minimum temperatures have shown a slight upward trend over time.

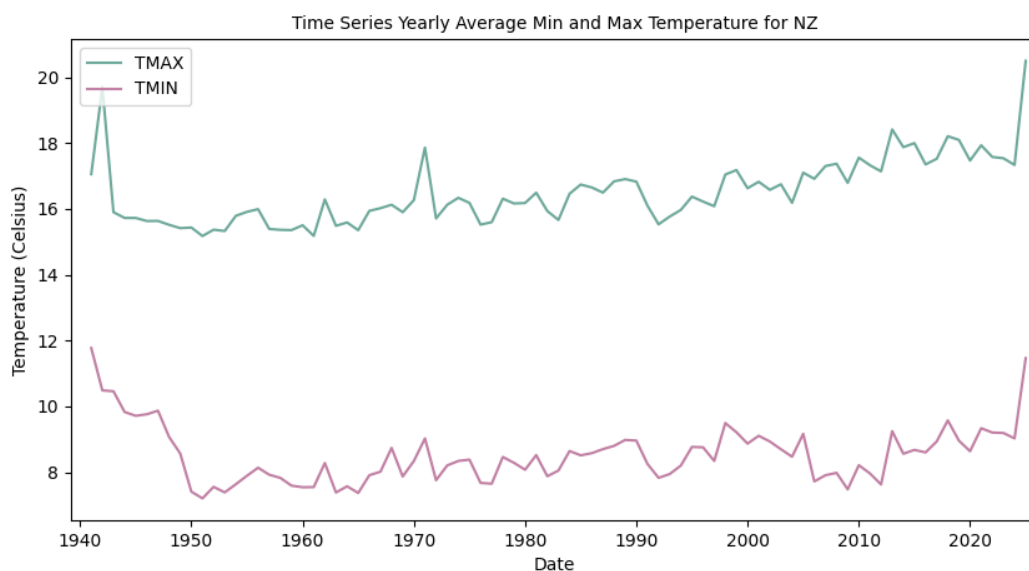


Figure 5 NZ yearly average minimum and maximum temperature over time

Visualizations Q2

(a) Calculation and Descriptive Statistics Average Annual Rainfall by Country

Rainfall observations were filtered from the daily dataset by selecting records where the “ELEMENT” column is equal to “PRCP,” focusing on precipitation analysis. The country code was extracted from the first two characters of the station ID, and the year was derived from the “DATE” column. The data was then grouped by both country code and year to calculate the average precipitation value for each group. The resulting dataset was cached for efficiency, as it will be used frequently in subsequent analyses, and it was also saved for future reference.

To generate descriptive statistics for the dataframe, the Spark DataFrame function `describe()` and the distinct count were applied. The results are as follows:

Summary / Feature	Country Code	Year	Average Rainfall (mm)
Count	17712	17712	17712
Distinct count	218	242	NA
Mean	NA	1969.24	4.18
Standard deviation	NA	42.01	8.42
Minimum	AC	1750	0.11
Maximum	ZI	2025	436.10

Table 4 Descriptive analysis of daily rainfall observations

Based on the descriptive statistics in Table 4, the aggregated dataframe contains 17,712 observations, covering 218 countries over 242 years, from 1750 to 2025. The minimum yearly average rainfall is approximately 4.18 mm, while the maximum reaches 436.10 mm. The difference between the maximum and minimum annual rainfall for a given country is nearly 436 mm. The summary results indicate that the average yearly rainfall across all countries is about 4.18 mm, with a variation of 8.42 mm.

Next, the top countries with the highest average annual rainfall in a single year were identified by sorting the aggregated dataframe in descending order of rainfall values. To enrich the analysis, the country names were added to the dataframe by joining it with the country reference dataframe on the “COUNTRY CODE.” The results for the top five countries are presented in the following table.

Country Name	Year	Average Rainfall (mm)
Equatorial Guinea	2000	436.10
Dominican Republic	1975	341.40
Laos	1974	248.05
Belize	1978	224.47
Sint Maarten	1979	196.70

Table 5 Top 5 countries with the highest average rainfall in a single year

As shown in Table 5, Equatorial Guinea — one of the countries closest to the equator — recorded the highest average annual rainfall of 436.1 mm in the year 2000. This is nearly 95 mm higher than the second-highest record, which was 341.3 mm in the Dominican Republic in 1975.

To identify outliers, the Z-score was calculated for the average rainfall values. Observations with an absolute Z-score (Prabhakaran, 2023) greater than 3 were classified as outliers. The results are presented as follows:

Category	Year	Average Rainfall (mm)	Country Name
Bottom 5	1958	29.51	New Caledonia [France]
Bottom 5	1975	29.59	New Caledonia [France]
Bottom 5	1975	29.87	Tuvalu
Bottom 5	1971	29.98	New Caledonia [France]
Bottom 5	1960	30.30	New Caledonia [France]
Top 5	1979	196.70	Sint Maarten
Top 5	1978	224.47	Belize
Top 5	1974	248.05	Laos
Top 5	1975	341.40	Dominican Republic
Top 5	2000	436.10	Equatorial Guinea

Table 6 Top 5 and bottom 5 countries by average rainfall in the outlier set

There are 121 records in the outlier set. Table 6 presents these outliers, with average rainfall values ranging from 29.51 mm to 436.10 mm, spanning the years from 1958 to 2000. Notably, French New Caledonia appears four times in the table — for the years 1958, 1960, 1971, and 1975 — which suggests the possibility of false positives. Being close to the equator, also supports the observation that countries near the equator tend to experience higher rainfall levels.

To validate the outlier findings, a histogram of the average rainfall values was plotted to visualise the overall distribution of the data, as shown below:

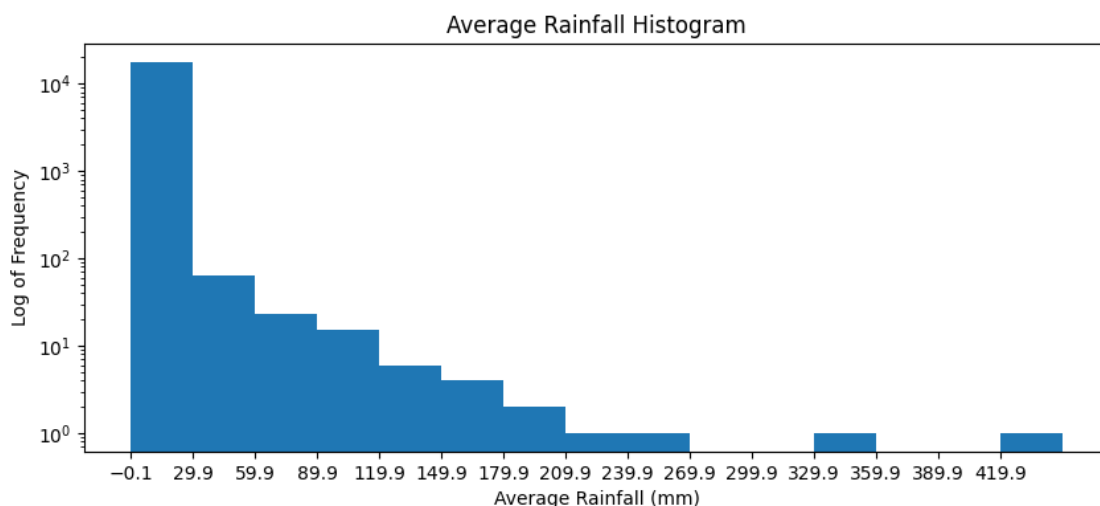


Figure 6 Average rainfall histogram

Due to the dominance of high-frequency bins in the original visualisation, which made it difficult to observe the other bins clearly, the logarithmic scale was applied to the frequency axis to better highlight the contrasts between bins. This adjustment confirmed that rainfall readings close to or exceeding 30 mm are indeed outliers. However, the dataset also contains negative average rainfall values, which are problematic.

By sorting the average rainfall dataframe in ascending order, it was found that the United Kingdom recorded a negative rainfall value of -0.11 mm in 1874. This clearly represents an outlier, as rainfall cannot be negative. Additionally, there are numerous rows with zero rainfall values, indicating a potential data quality issue. In total, there are 171 records with rainfall readings equal to or less than zero.

To further assess the reliability of the results, the dataframe was grouped by country, and the number of years with recorded rainfall data was counted for each country.

Country Name	Year Count
Dominica	3
Saint Lucia	4
Palmyra Atoll [United States]	5
Italy	218
Czech Republic	222
Germany	238

Table 7 Top 3 and bottom 3 countries by rainfall reading year count

Table 7 illustrates that some countries, such as Dominica and Saint Lucia, have only a few years of rainfall data, while Germany and Italy recorded rainfall over more than 200 years.

Figure 7 provides a closer look at the number of years for which rainfall data was collected. Most countries have data spanning between 20 and 90 years. As a result, the dataset is incomplete, with many missing values.

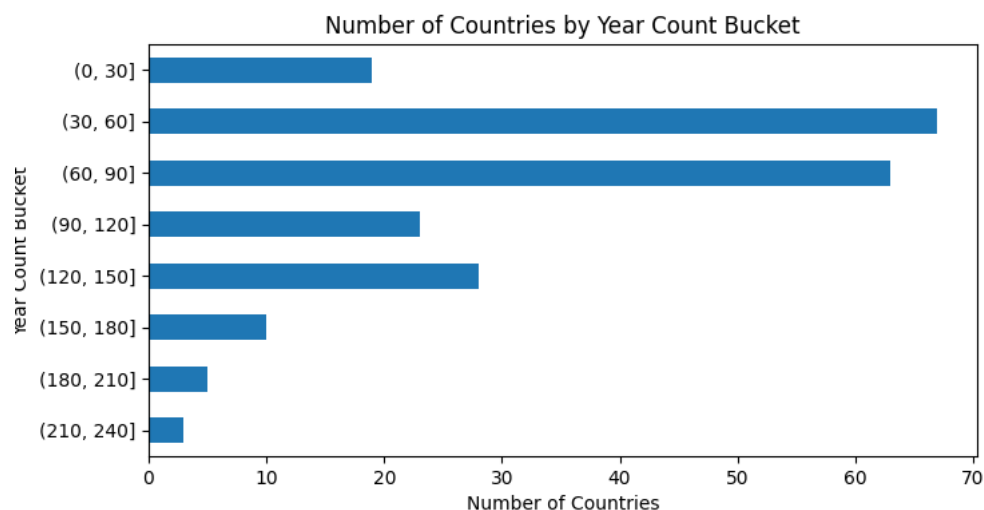


Figure 7 Number of countries by year count bucket

(b) Visualisation and Data Gap Analysis

The year 2024 was chosen for the world map visualisation by filtering the dataframe for “YEAR” 2024. The Choropleth map was imported to visualise the data.

The Choropleth map requires country codes in a 3-digit ISO format, while the rainfall data contains only 2-digit country codes. To resolve this, the pycountry Python library was installed and imported to map the 2-digit country codes to the corresponding 3-digit codes. However, discrepancies arose between the 2-digit country codes in the rainfall dataframe and the pycountry dictionary. As a result, the country name was also used to find the corresponding 3-digit country code in the pycountry dictionary. The country names were matched either against the official or common names in pycountry (e.g., “Vietnam” is listed as “Viet Nam” in pycountry).

The first attempt resulted in 49 countries failing to be mapped using pycountry. It was then discovered that some country names had extra spaces or discrepancies that prevented a match by either code or name. To address this, the extra spaces were removed from the country name column, and a dictionary was created to manually rectify the unmatched countries’ 2-digit codes, which were obtained via a Google search.

Finally, all countries were successfully plotted on the world map, as shown below:

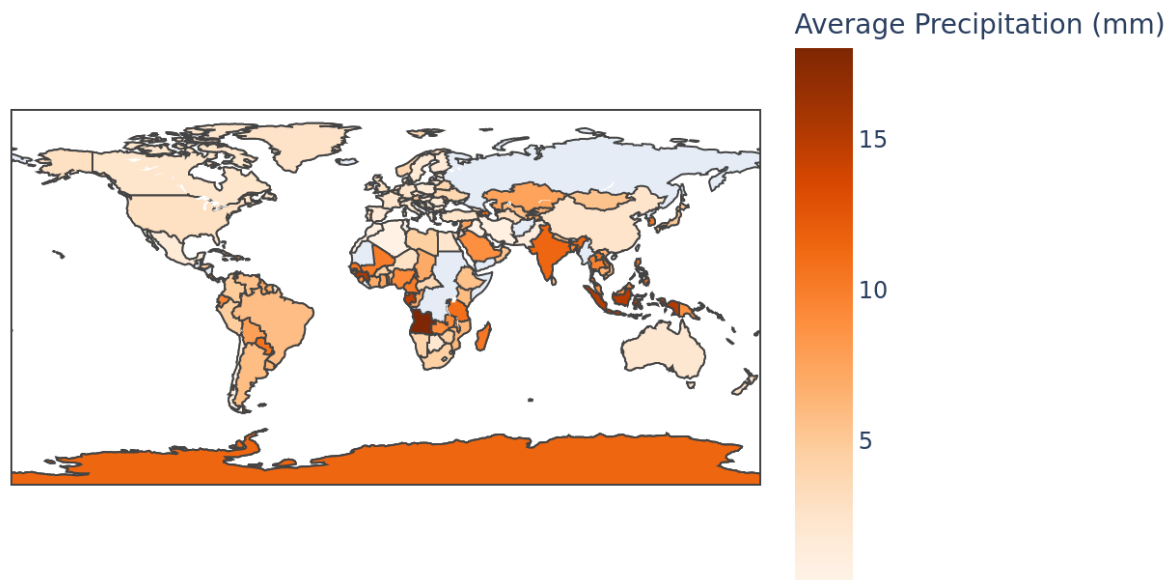


Figure 8 Average rainfall in 2024 world map

The world map in Figure 8 shows that countries closer to the equator (the central part of the map) generally have higher rainfall readings than those farther away. However, some countries’

data is missing, such as Russia, which appears as a blank grey continental area in the top right corner of the map.

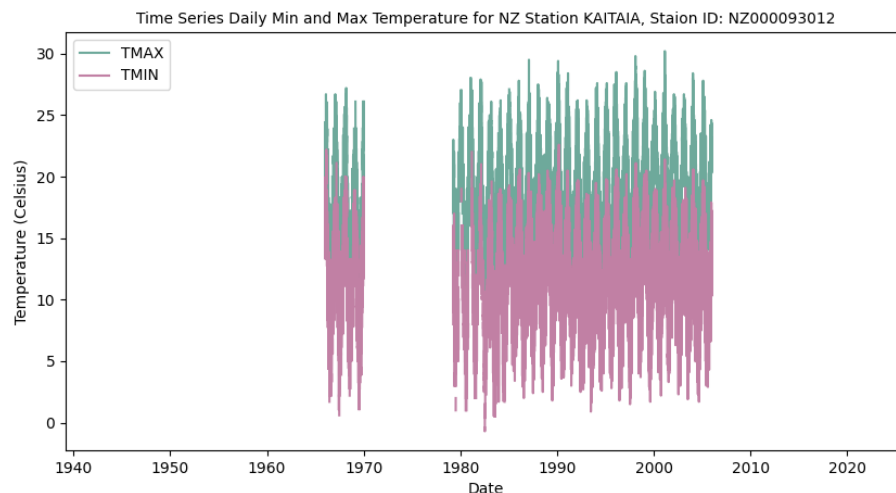
To determine how many countries are missing rainfall data in 2024, the pycountry dictionary was converted into a dataframe and then joined with the rainfall dataframe to find the rows that are missing rainfall readings. The result indicates that 79 countries worldwide are missing rainfall observations for 2024.

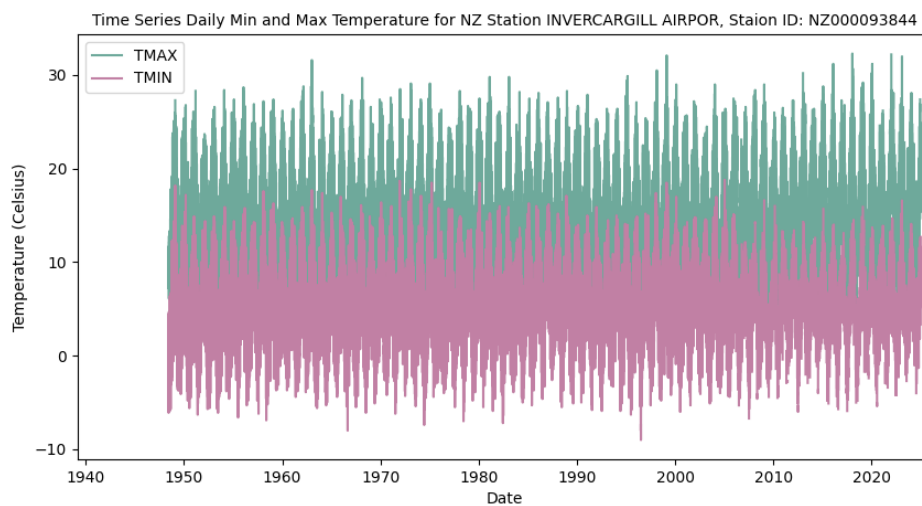
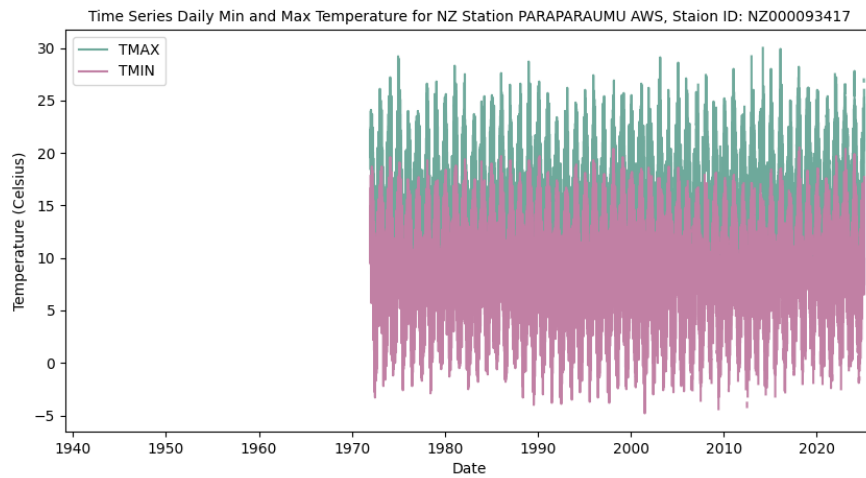
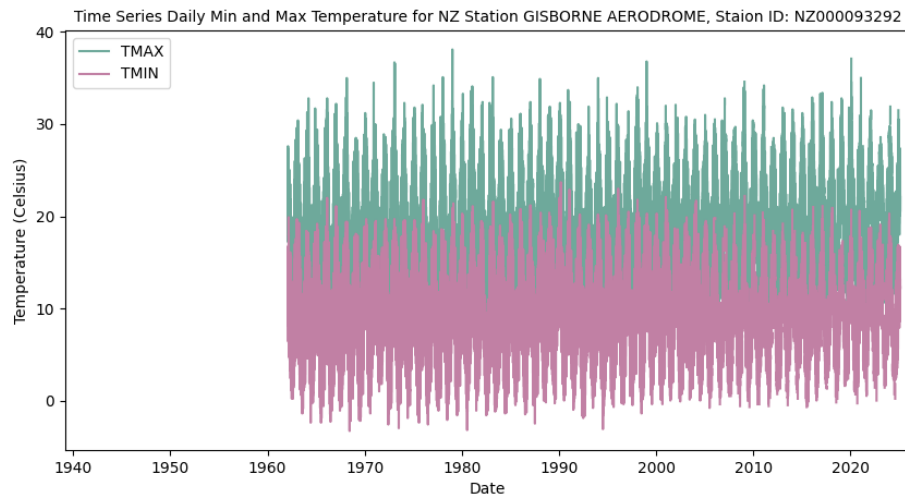
Conclusions

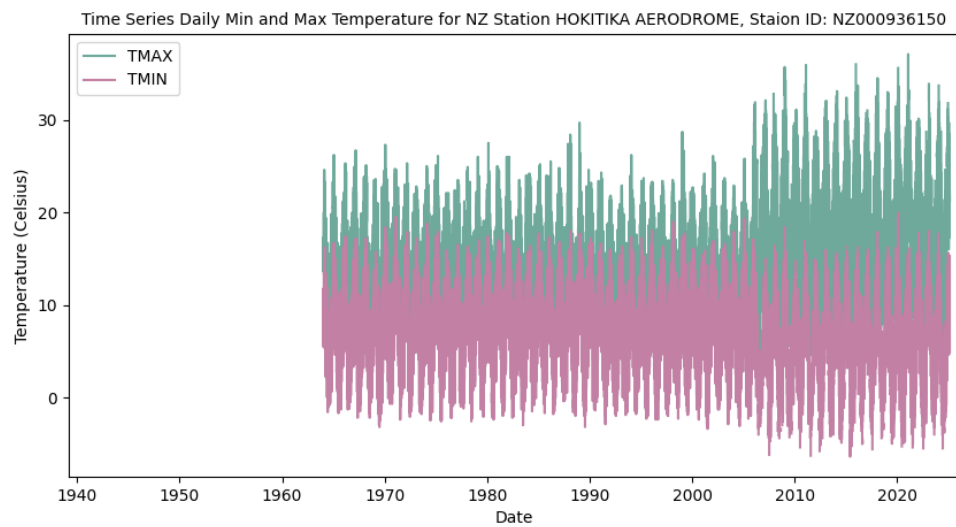
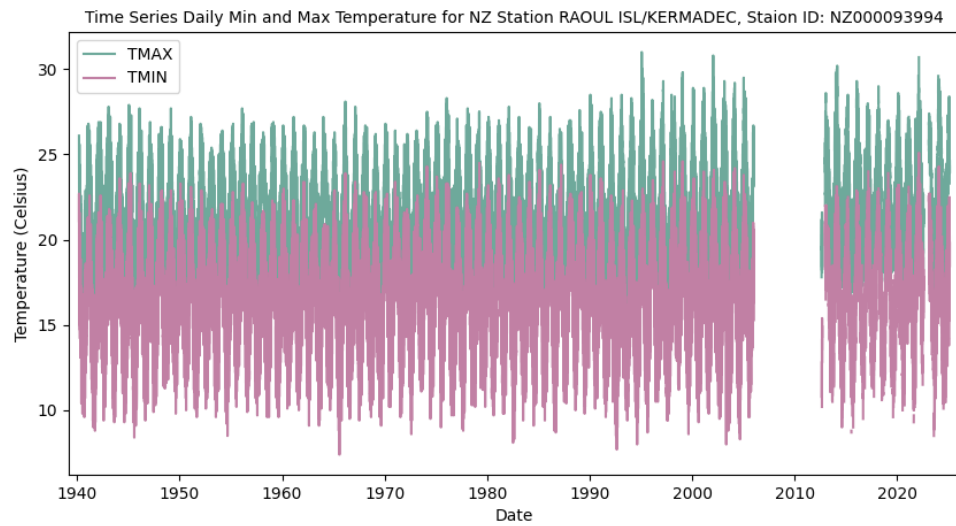
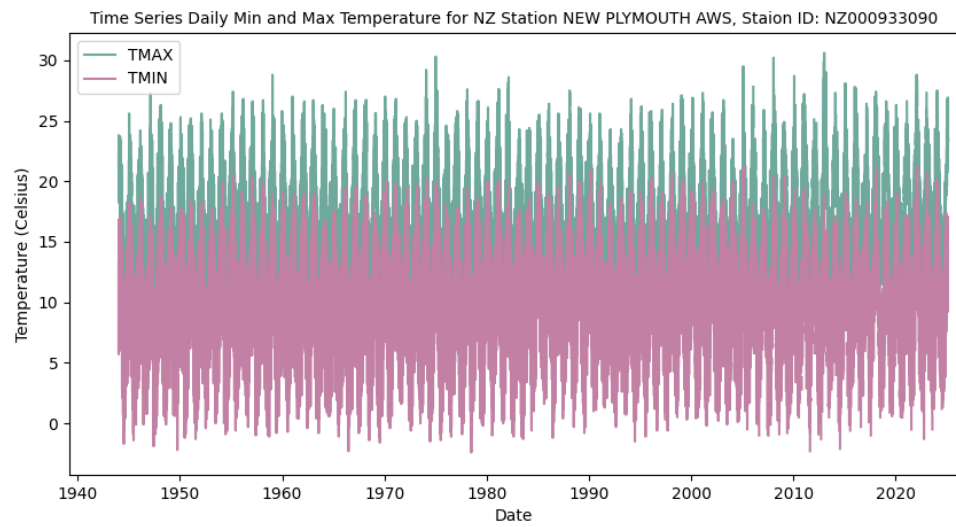
Spark efficiently handles large amounts of data, such as gigabytes, which would typically take much longer to process on a single local computer. It leverages the advantages of cloud computing by clustering computing resources to process data in a distributed and efficient manner, with the ability to scale up or down seamlessly based on demand. Additionally, the data stored in HDFS is replicated across nodes to ensure resilience.

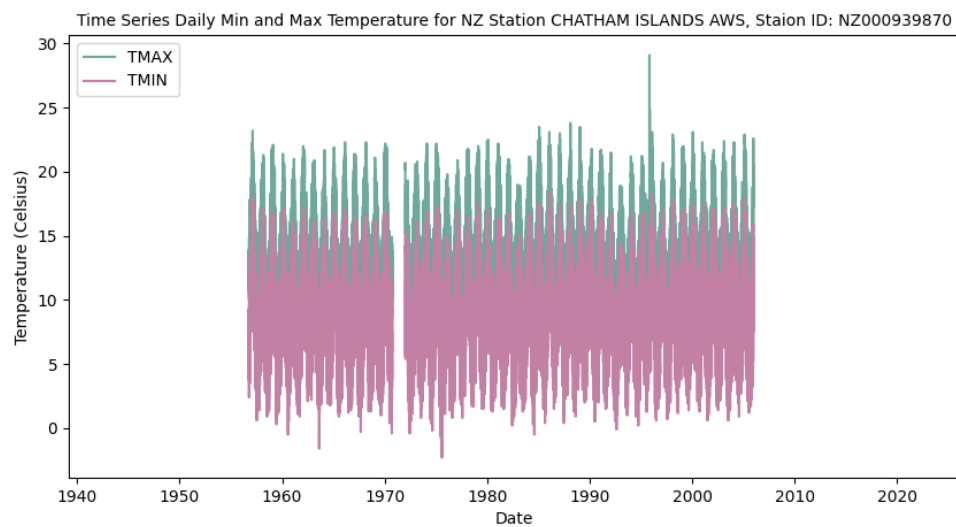
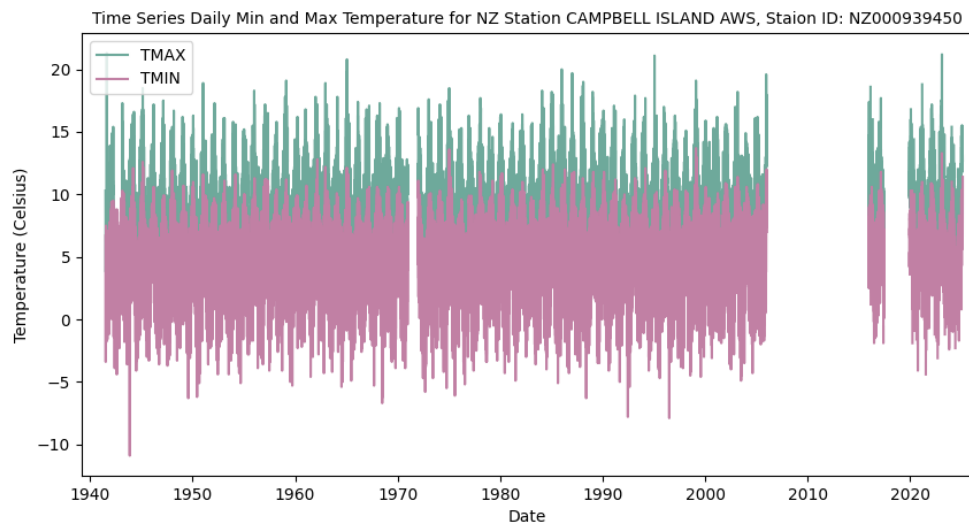
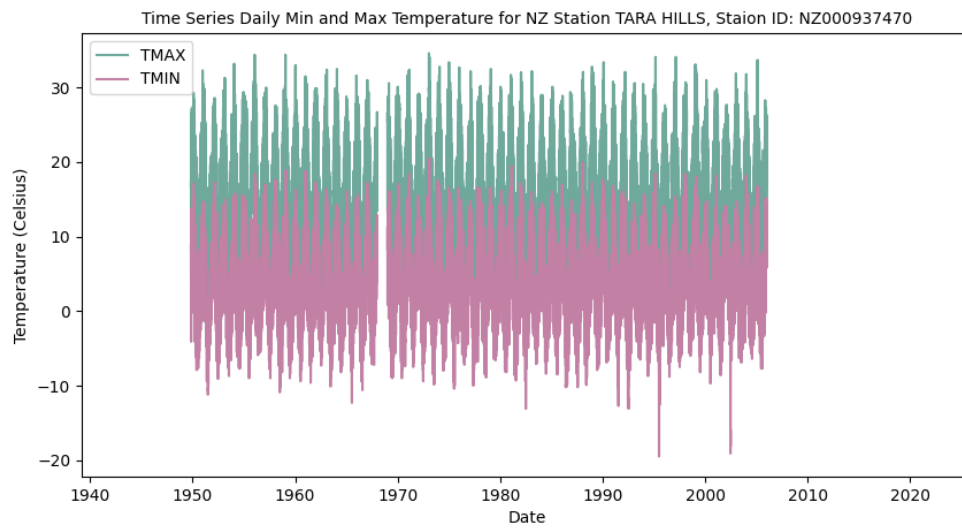
The GHCN dataset provides comprehensive climatology data from many countries and spans several years of data collection, supported by a broad network of stations. However, data quality must be prioritised to ensure accuracy, completeness, and consistency. While data quality has improved over time, issues persist in certain territories and countries, especially in recent years, when analysing temperature and precipitation data.

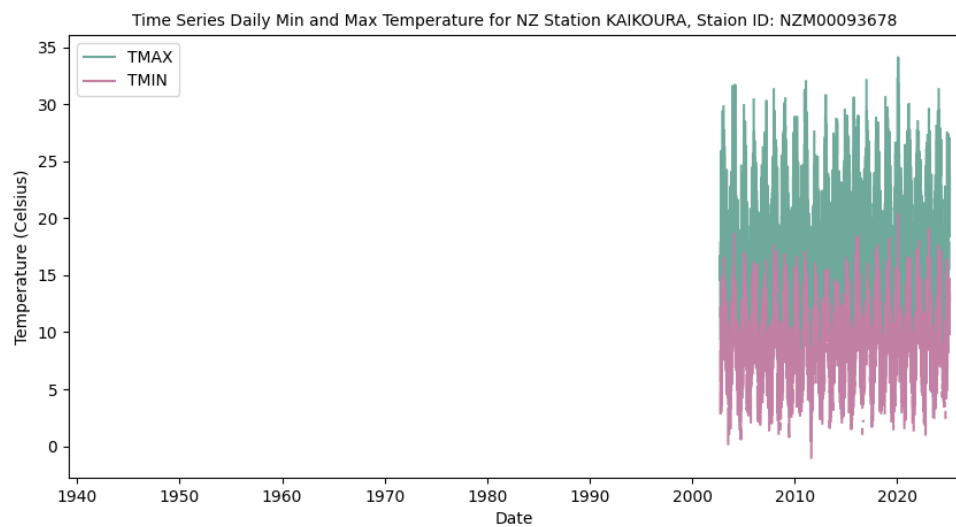
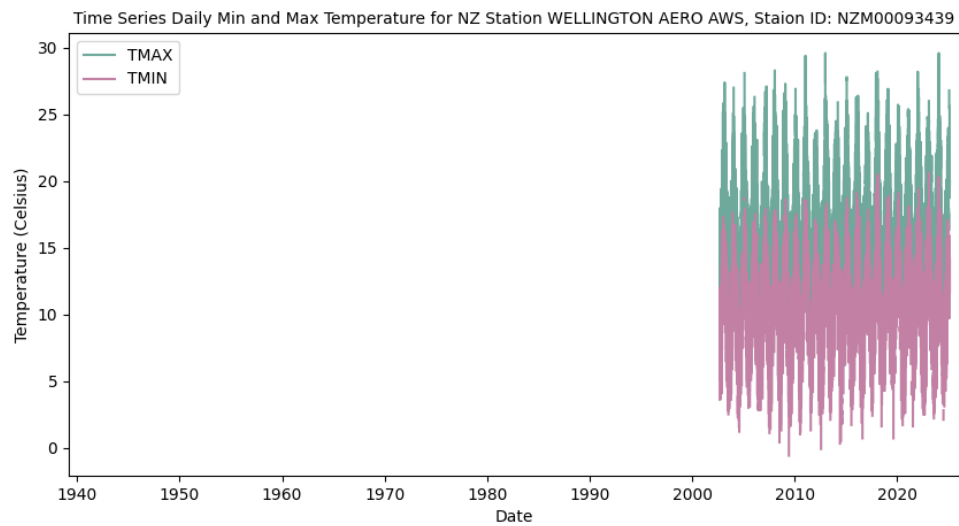
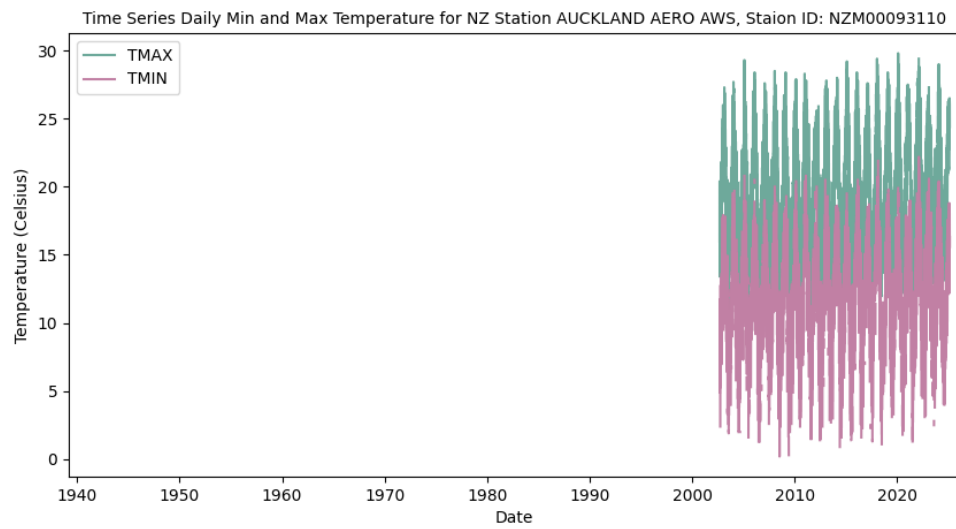
Appendices

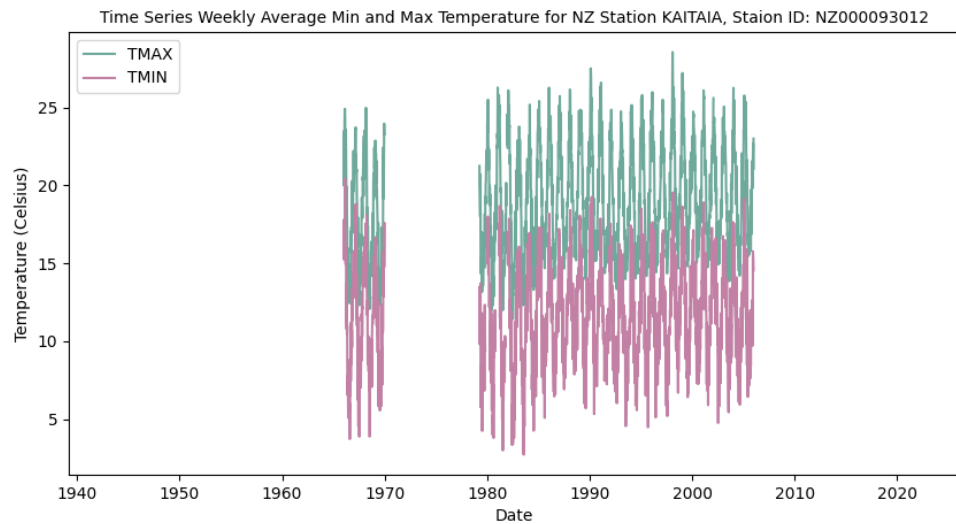
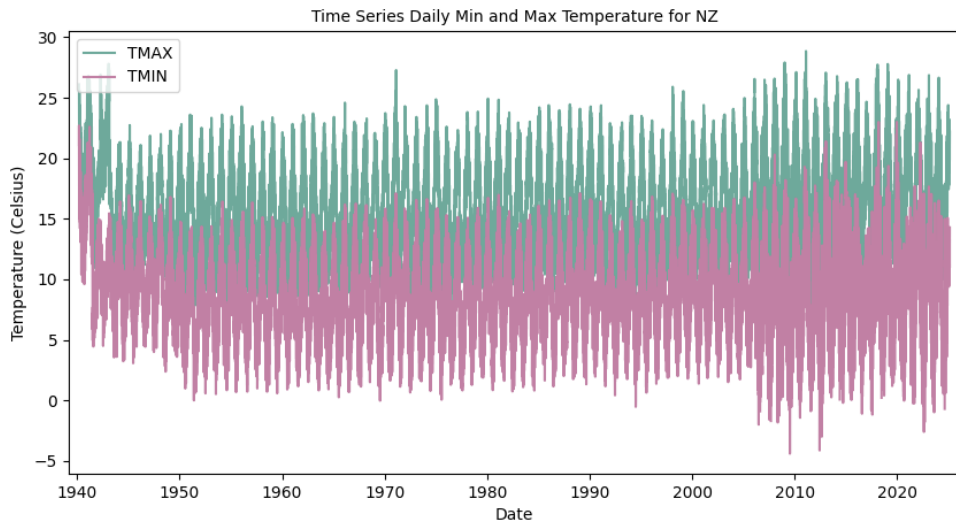
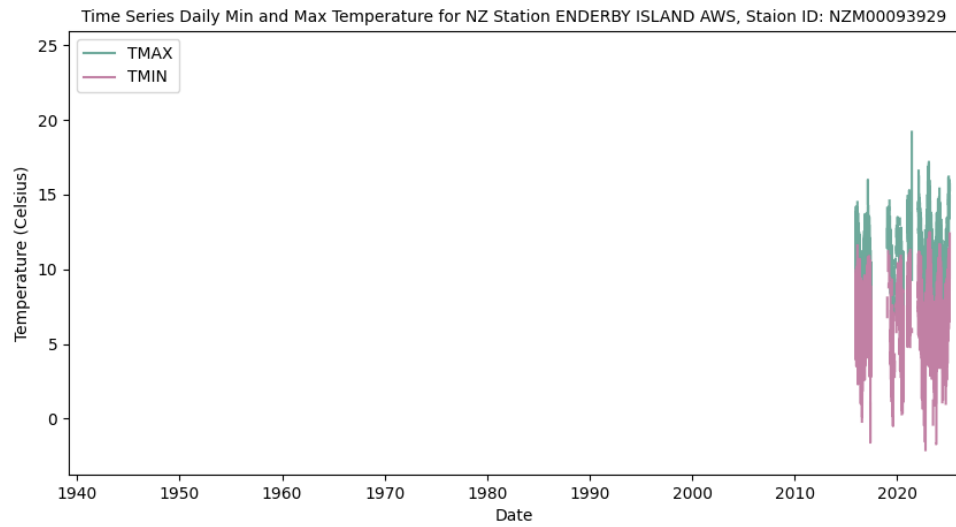


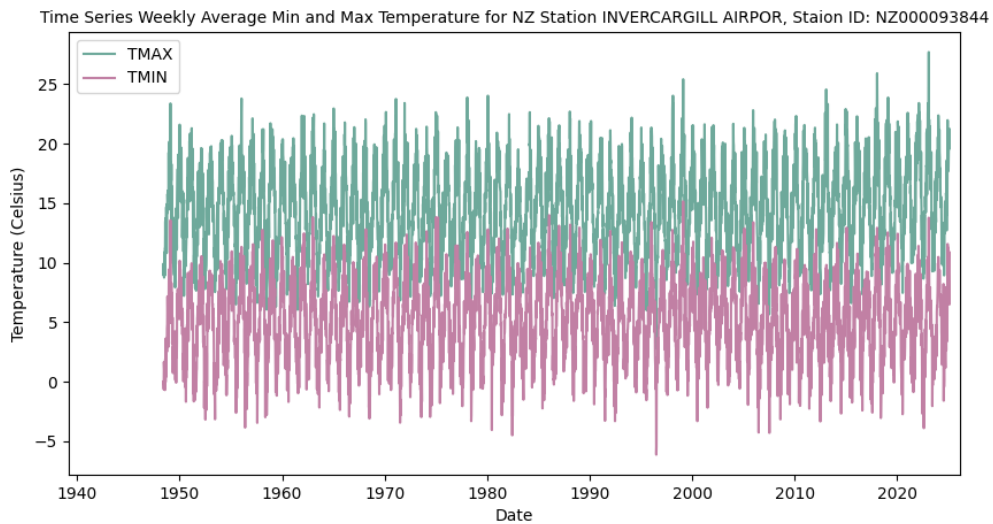
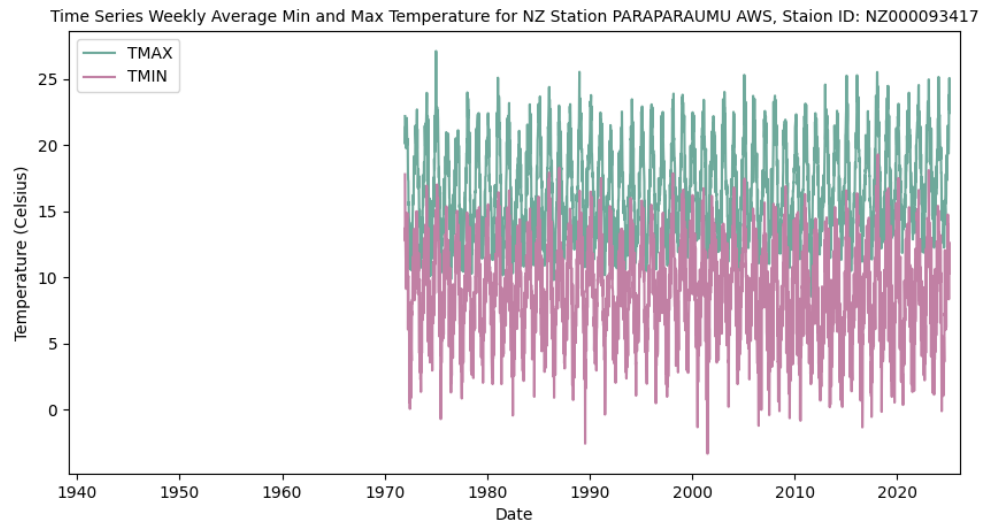
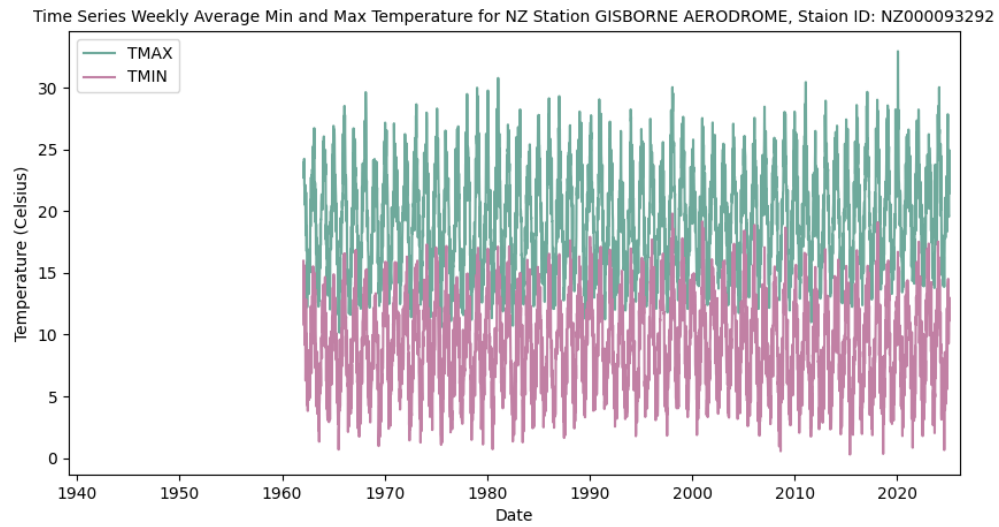


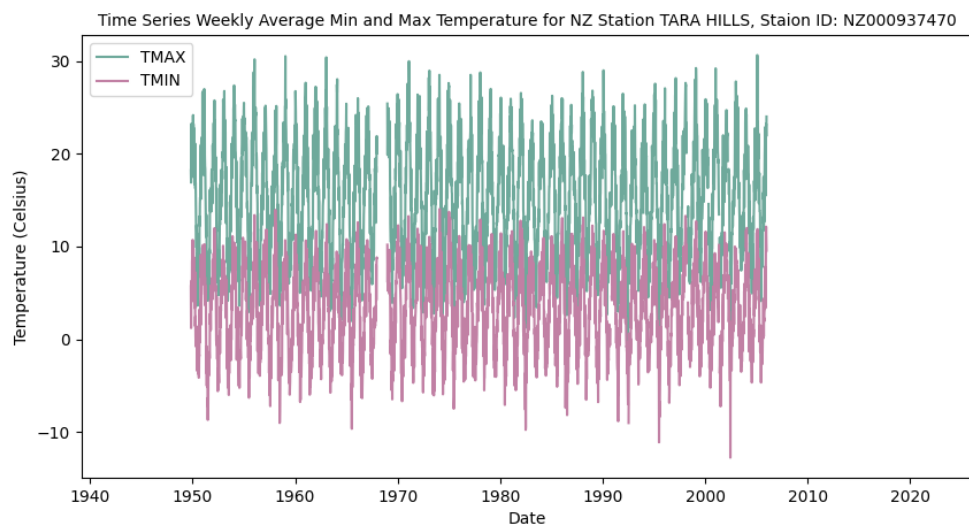
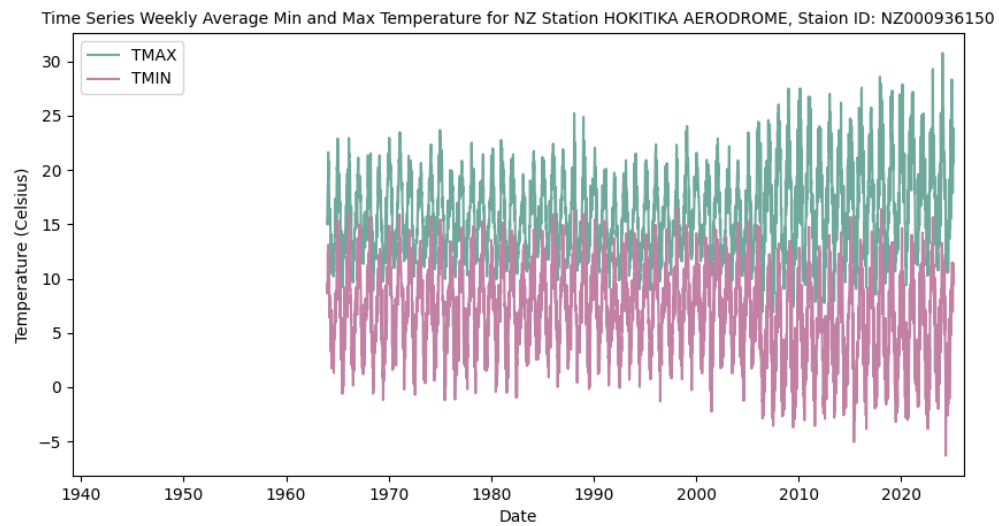
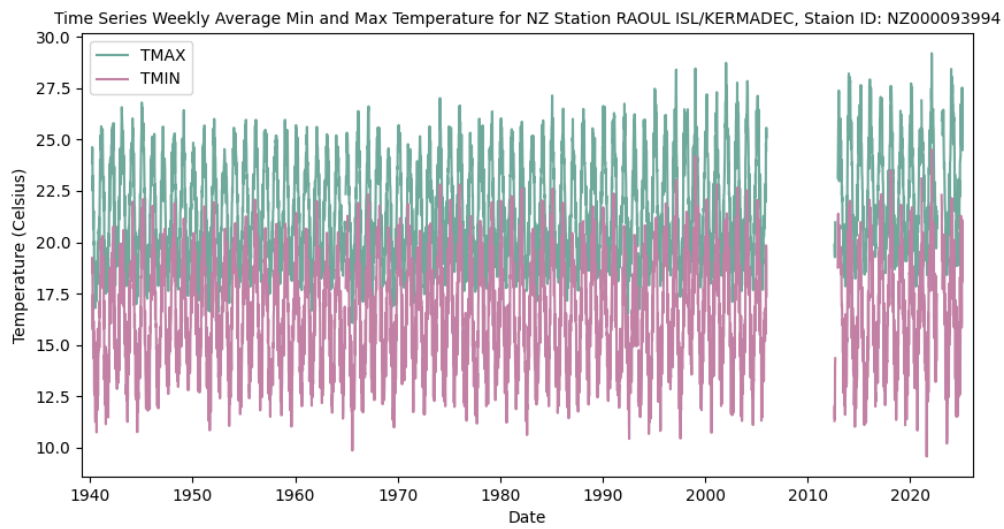


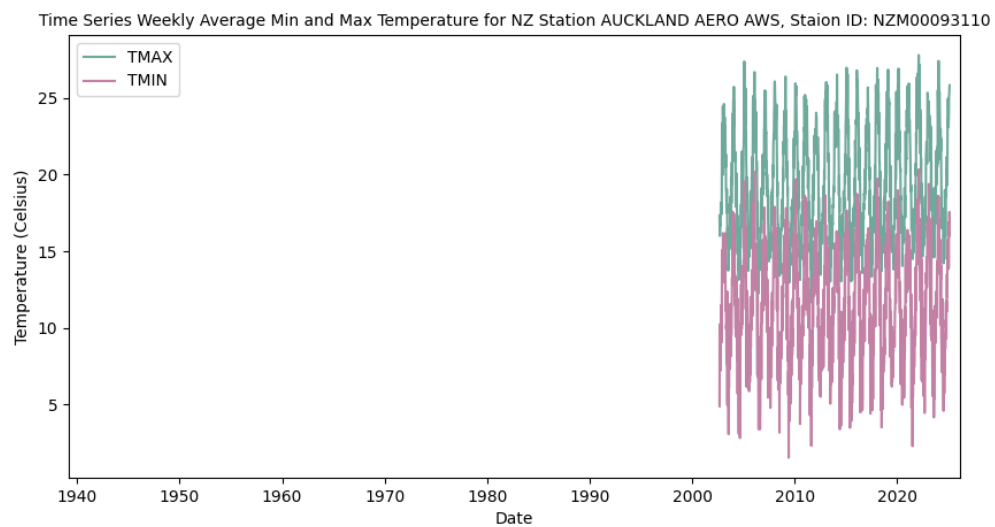
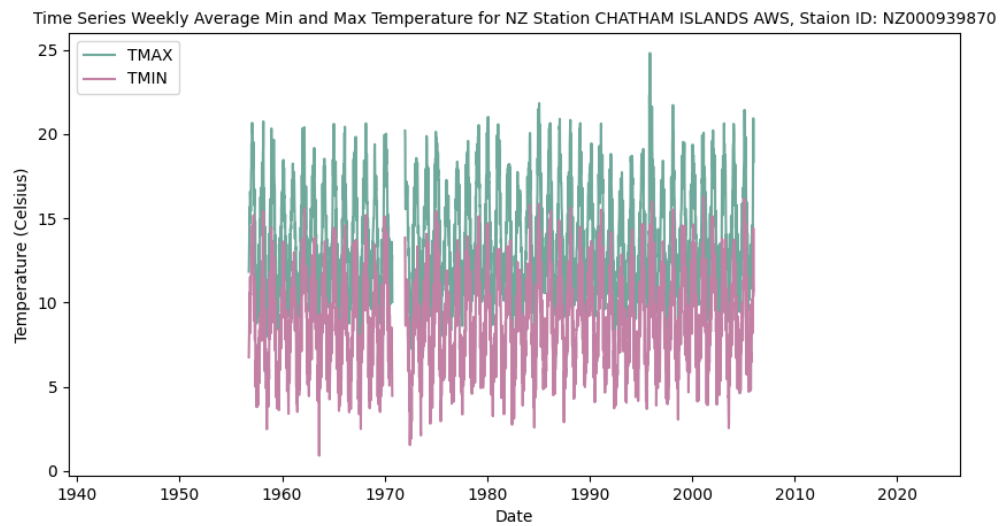
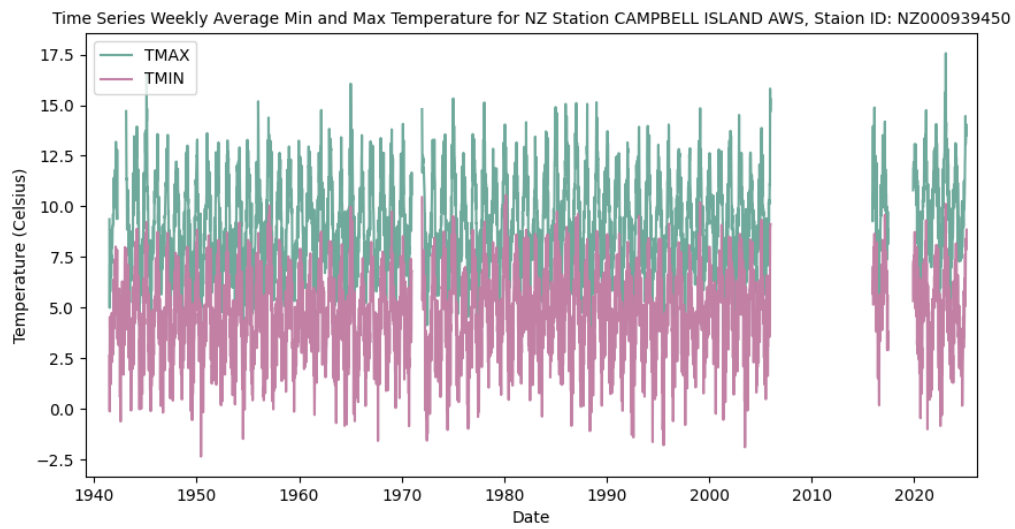


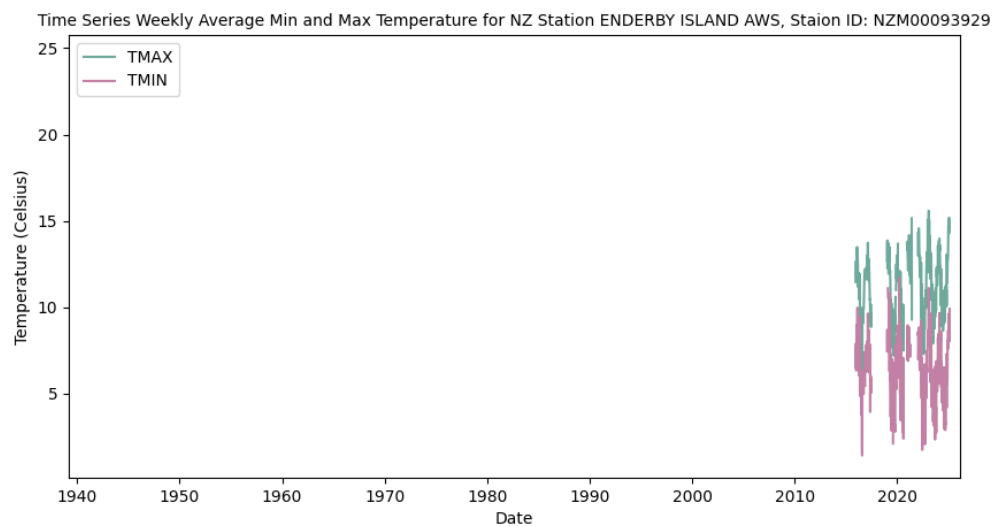
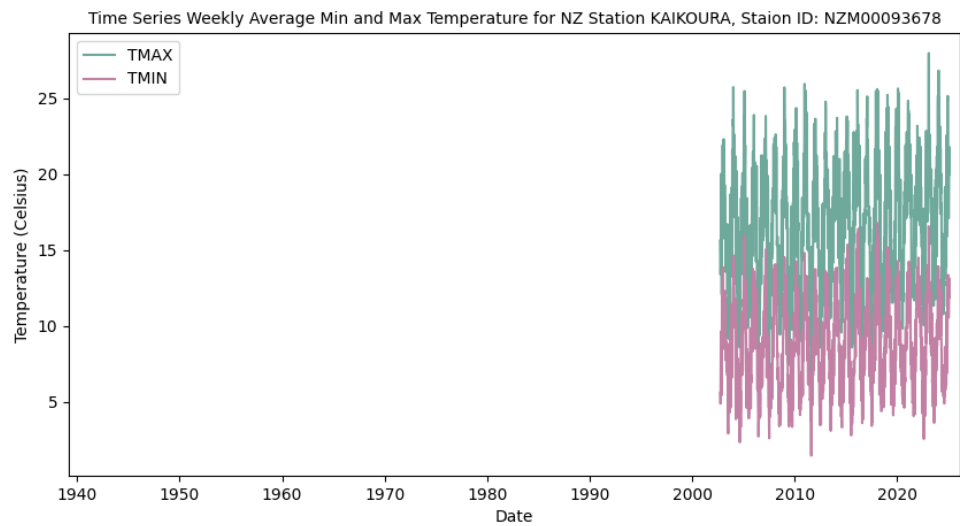
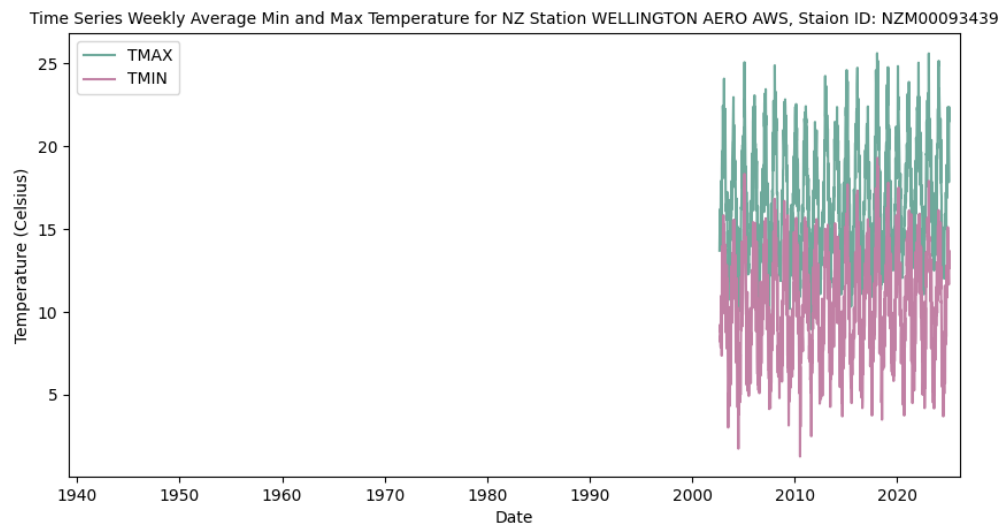


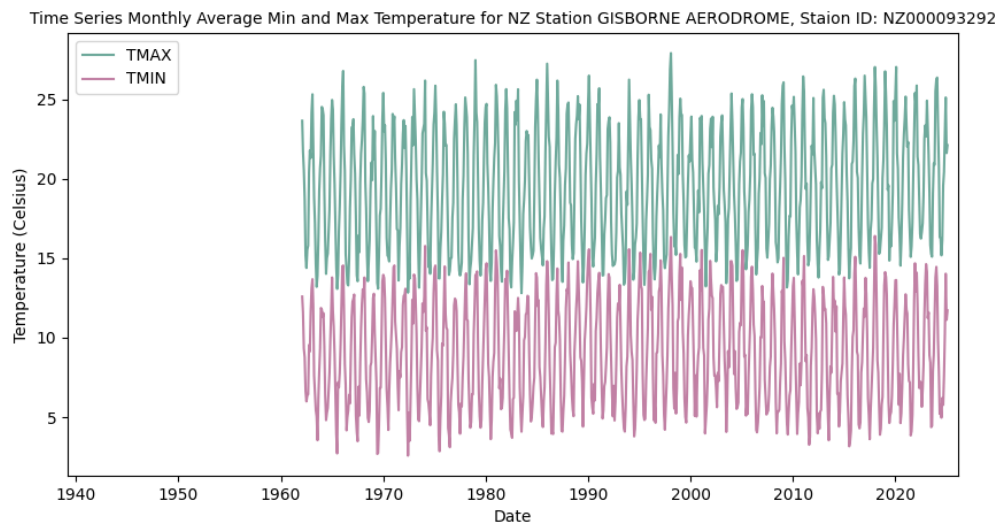
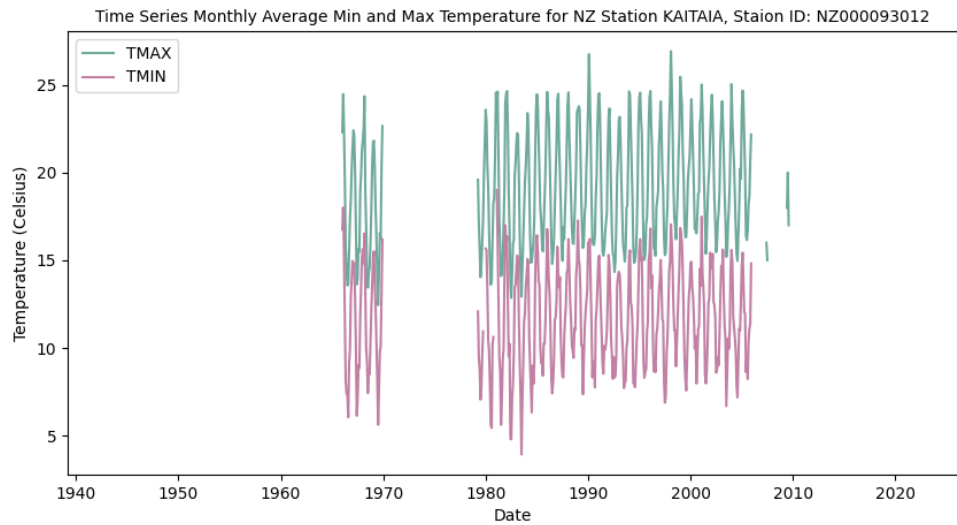
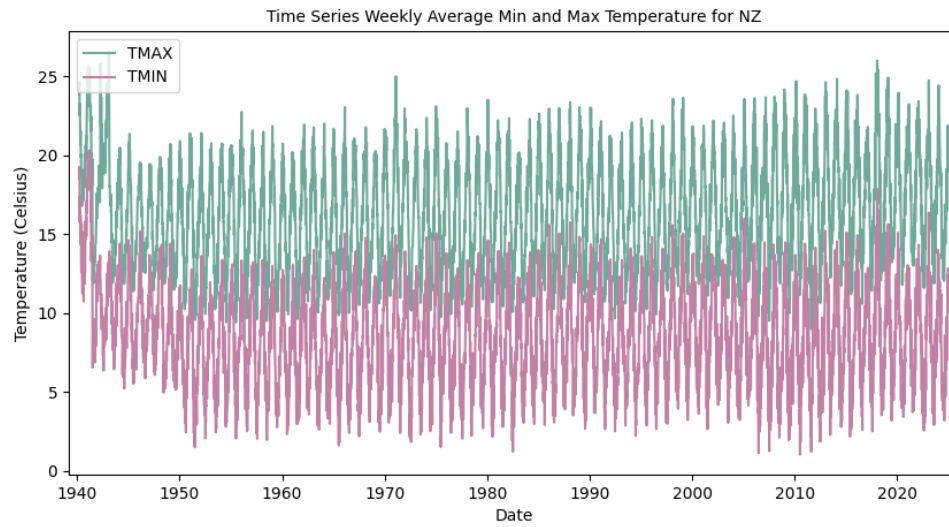


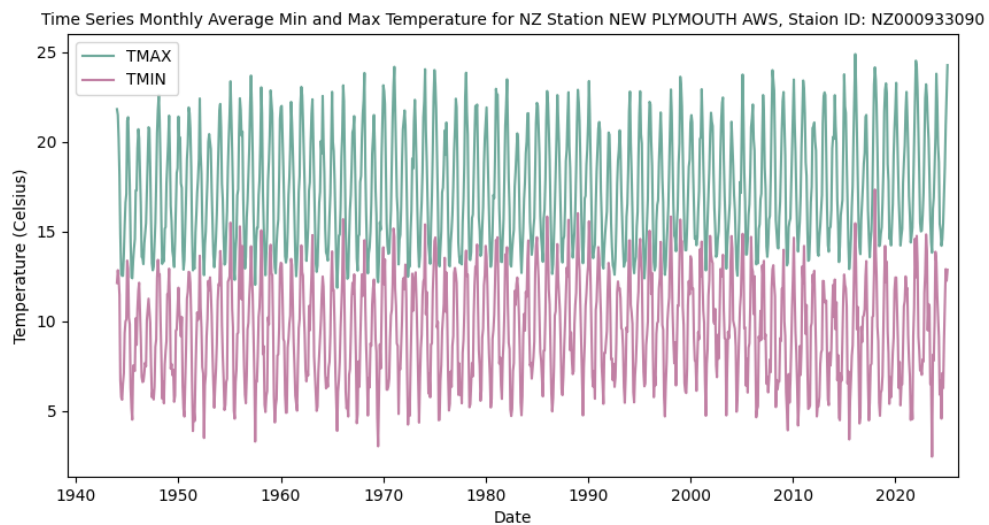
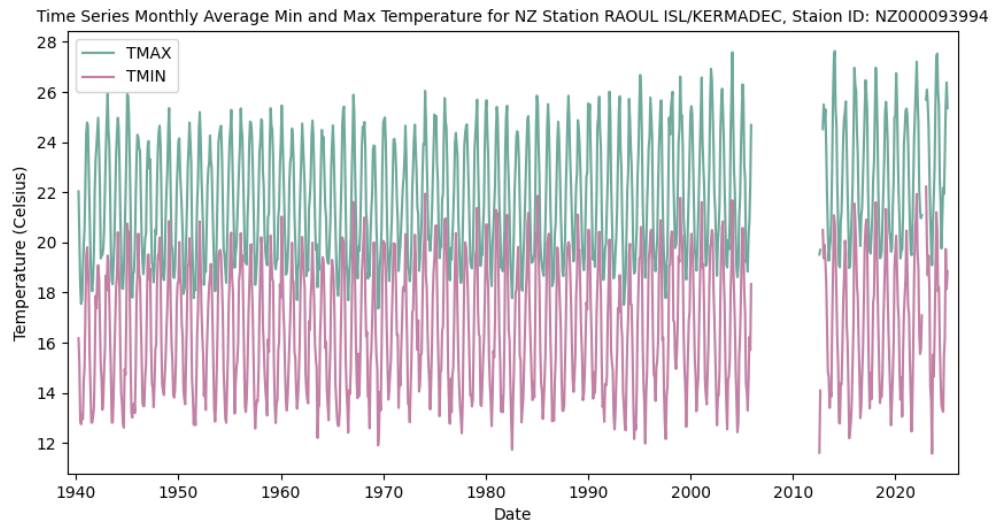
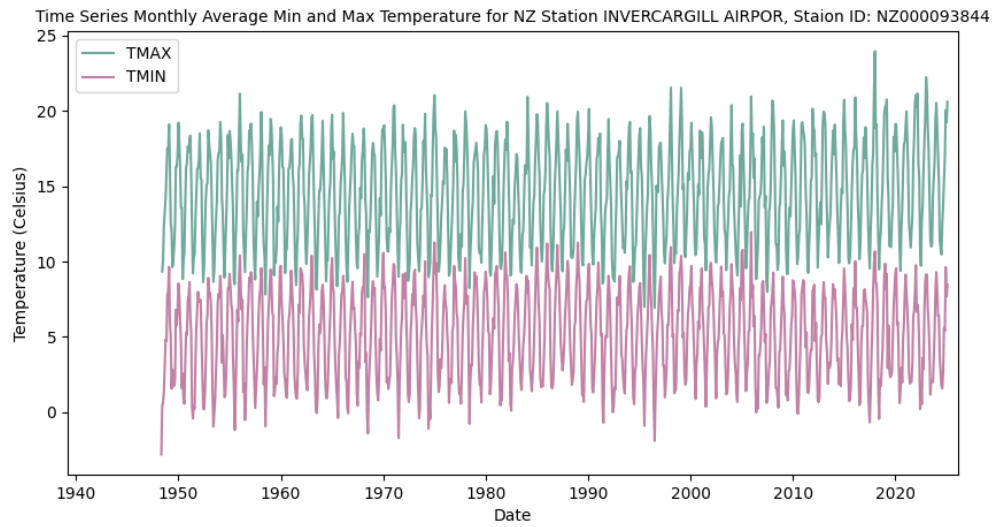


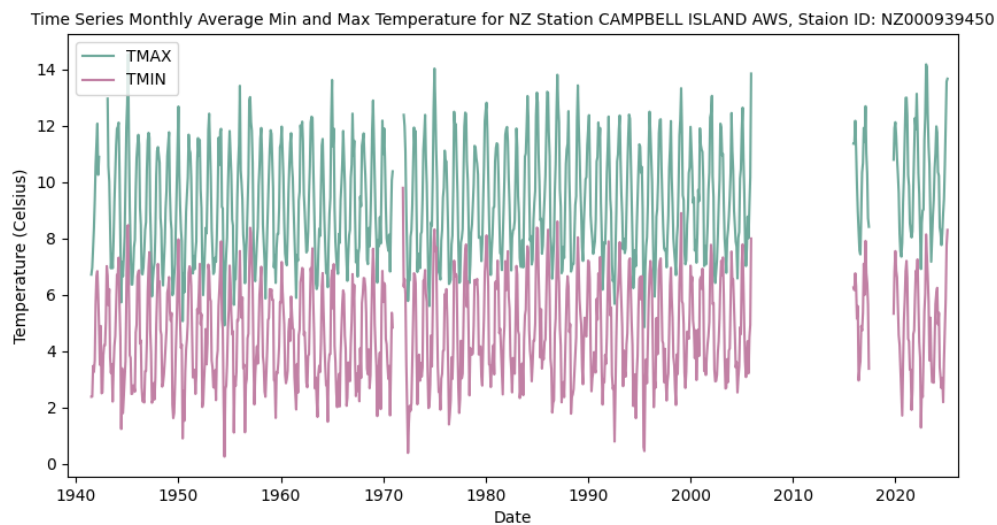
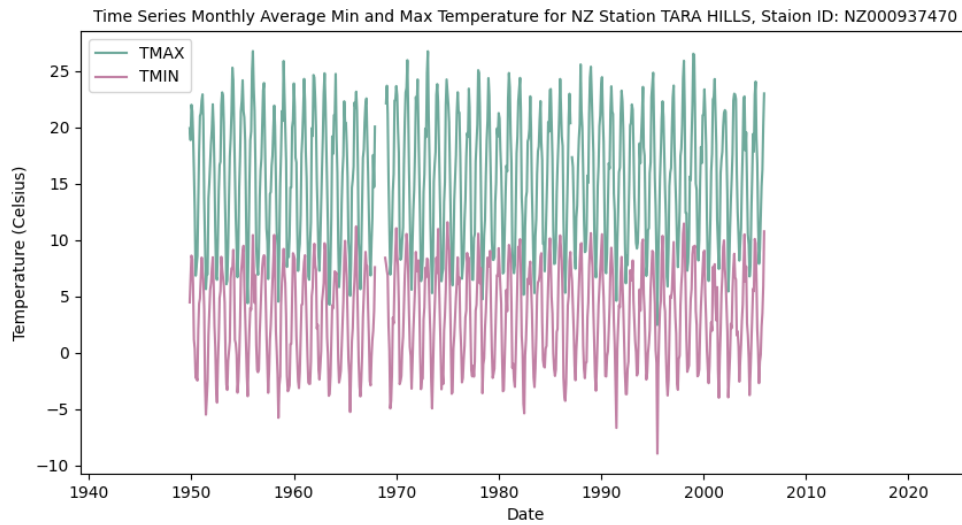
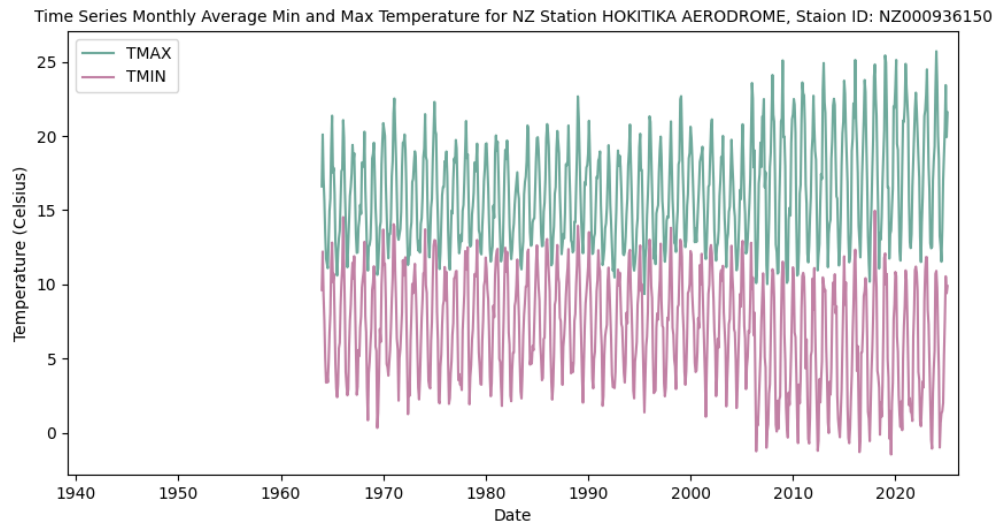


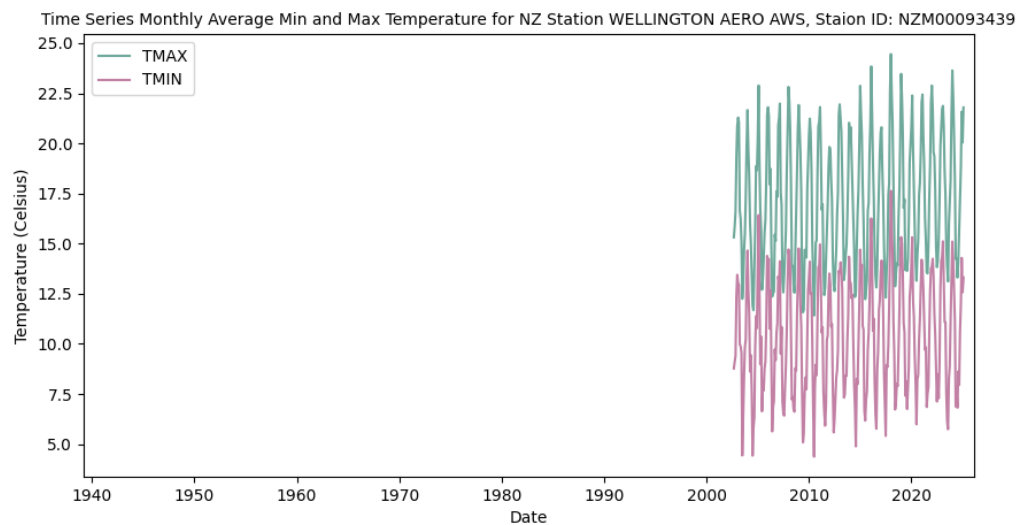
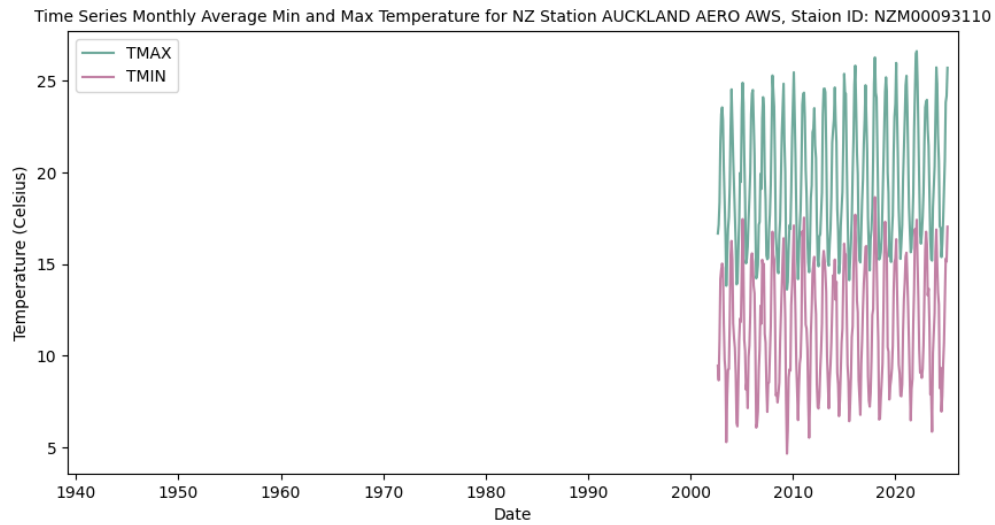
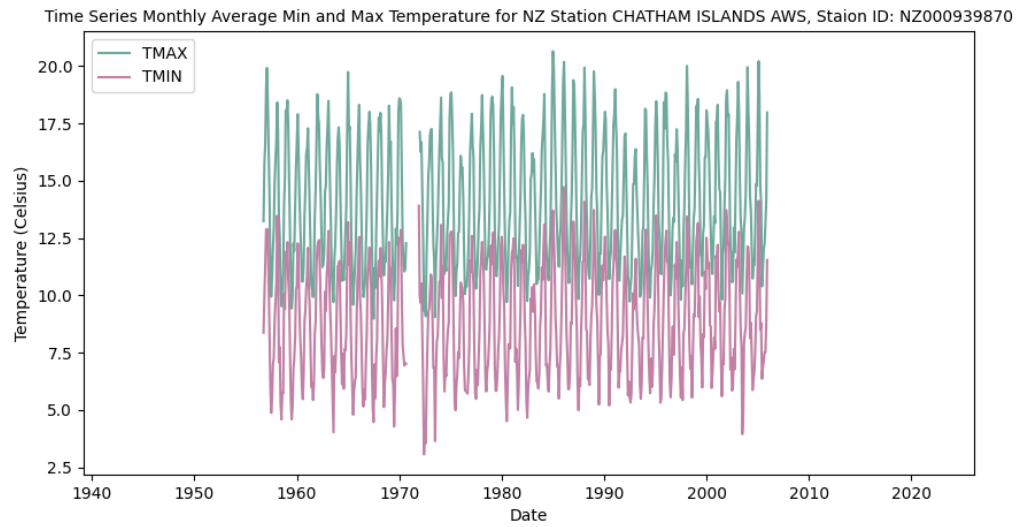


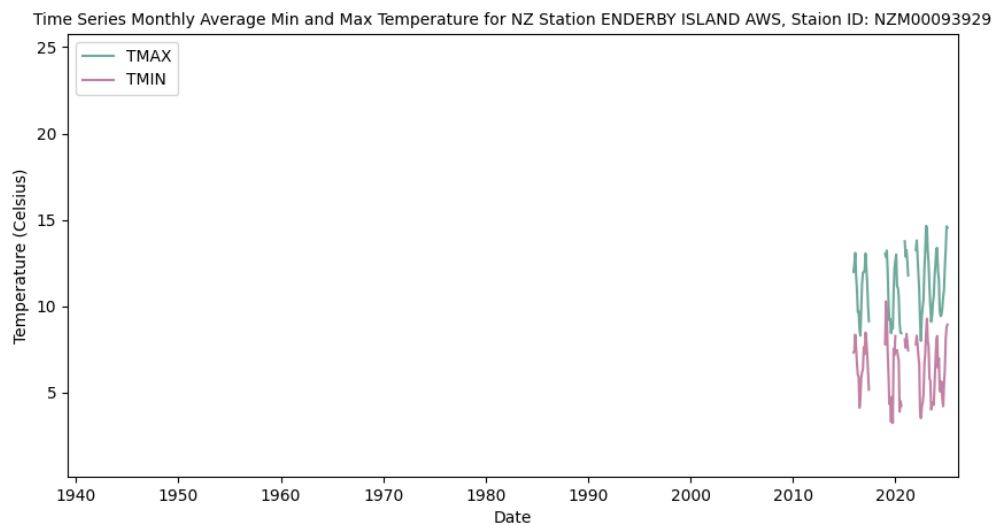
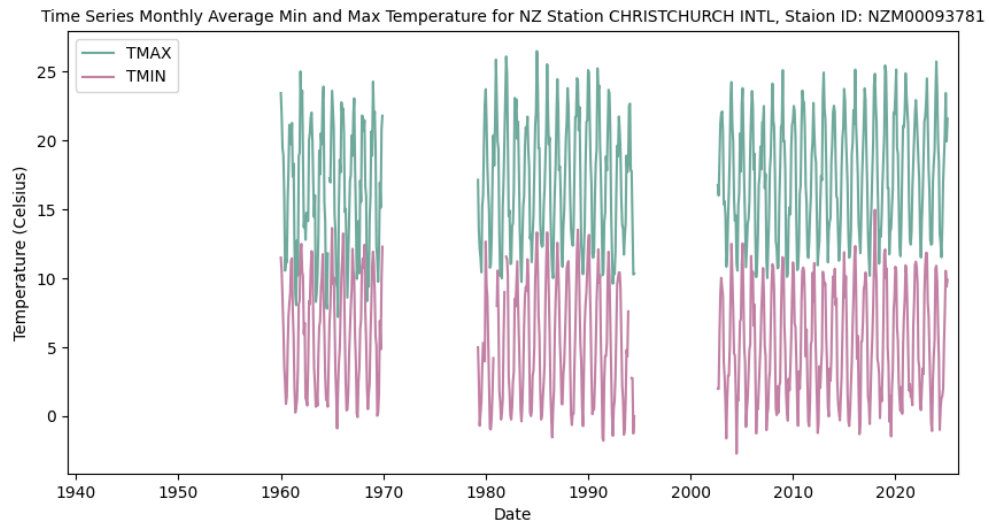
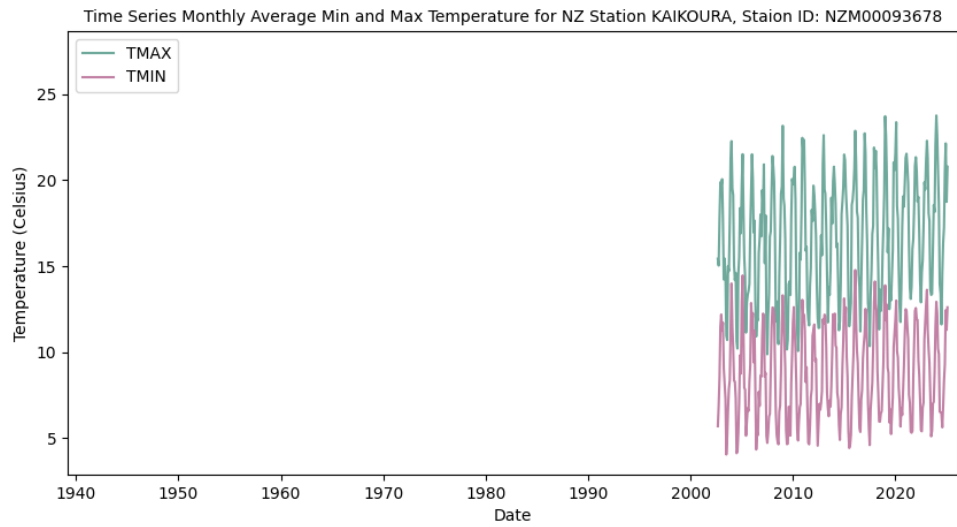


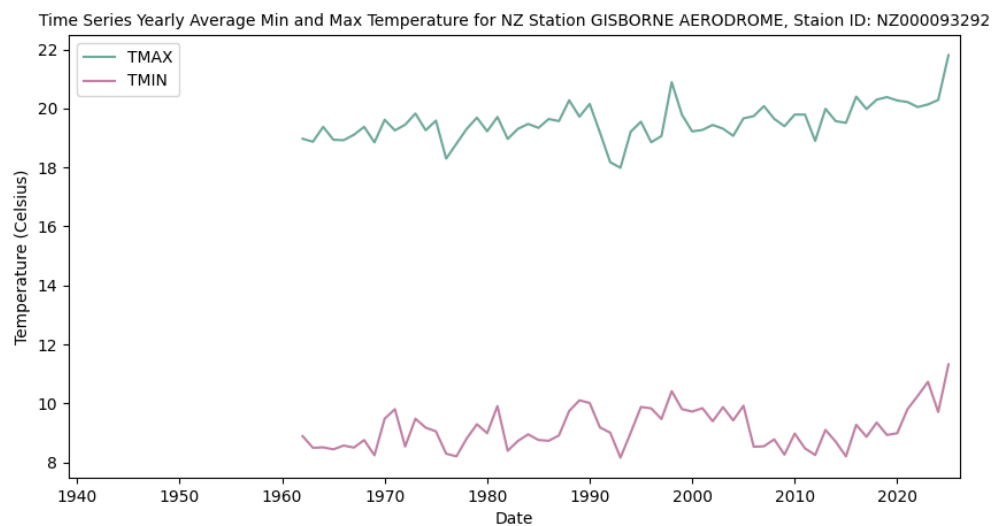
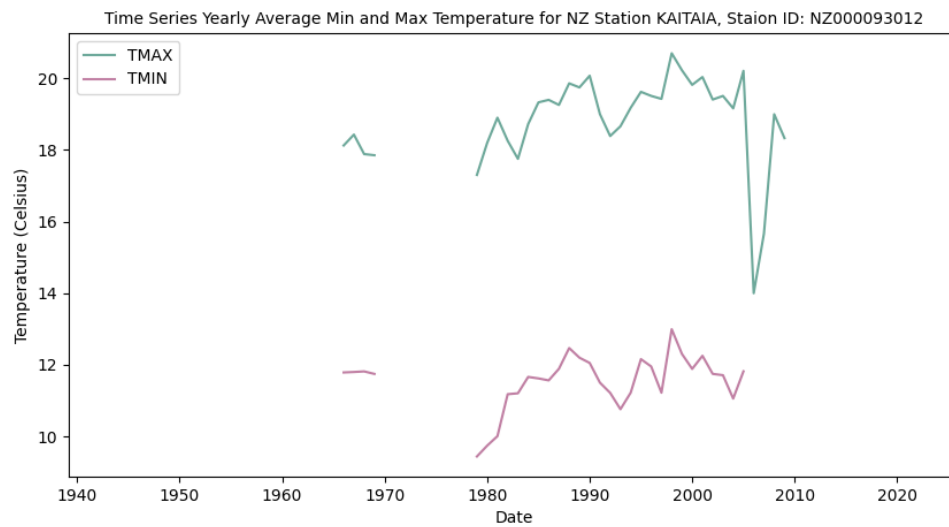
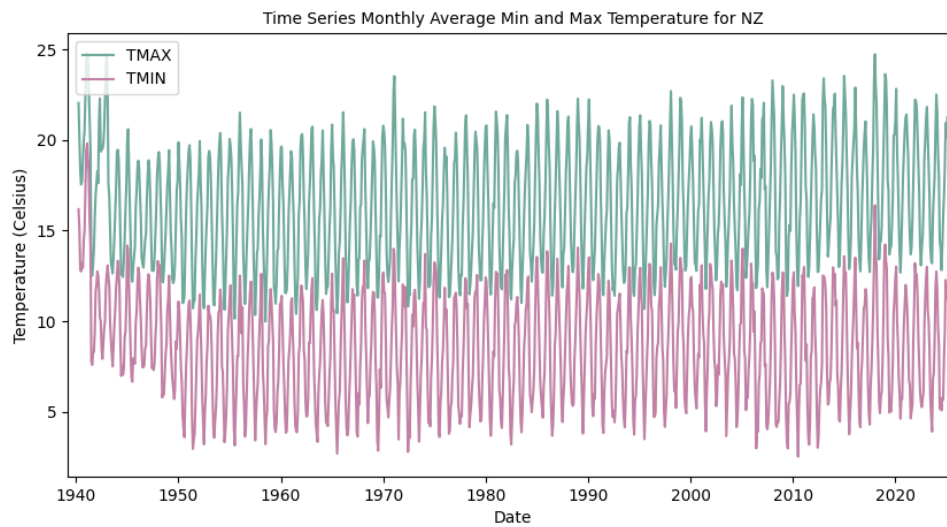


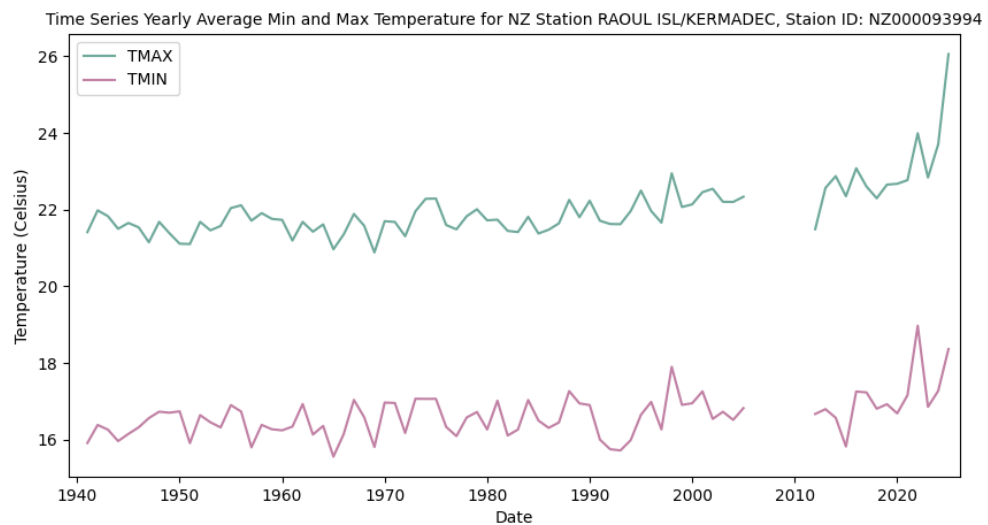
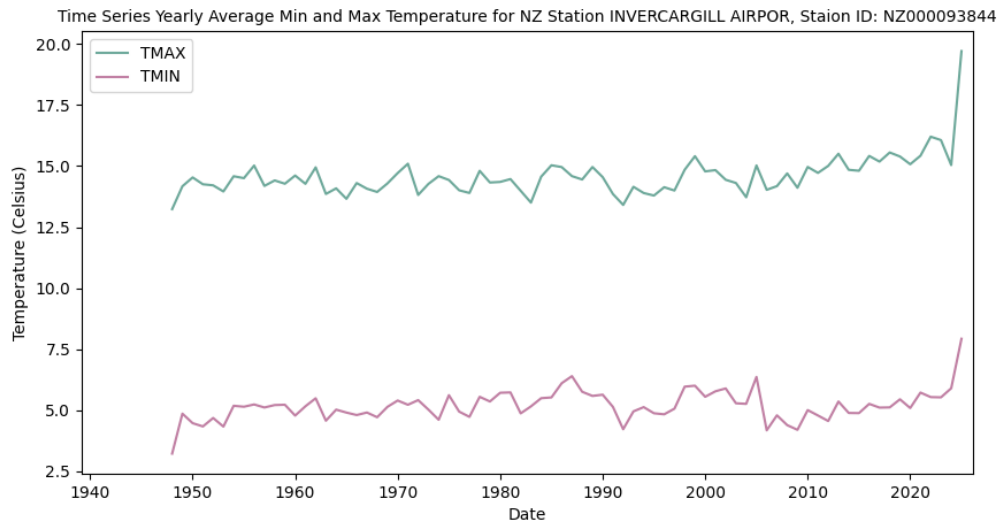
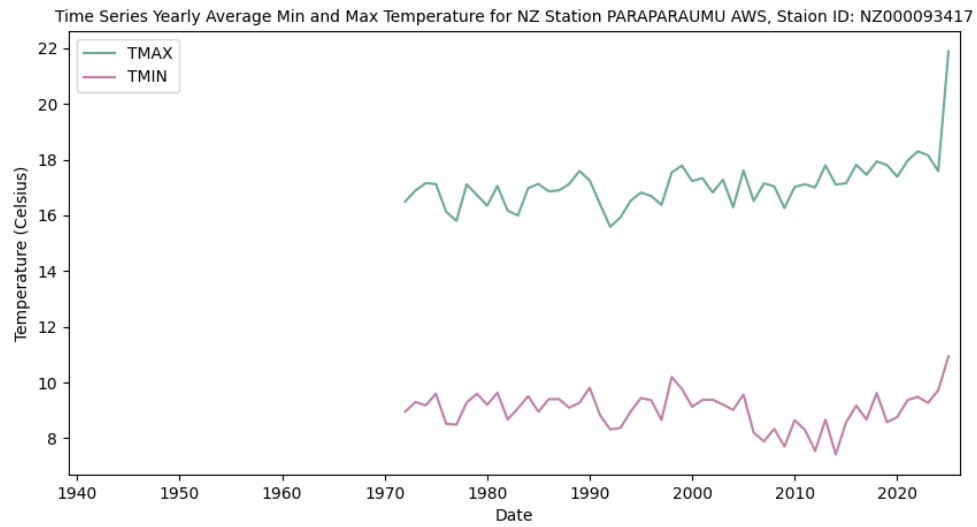


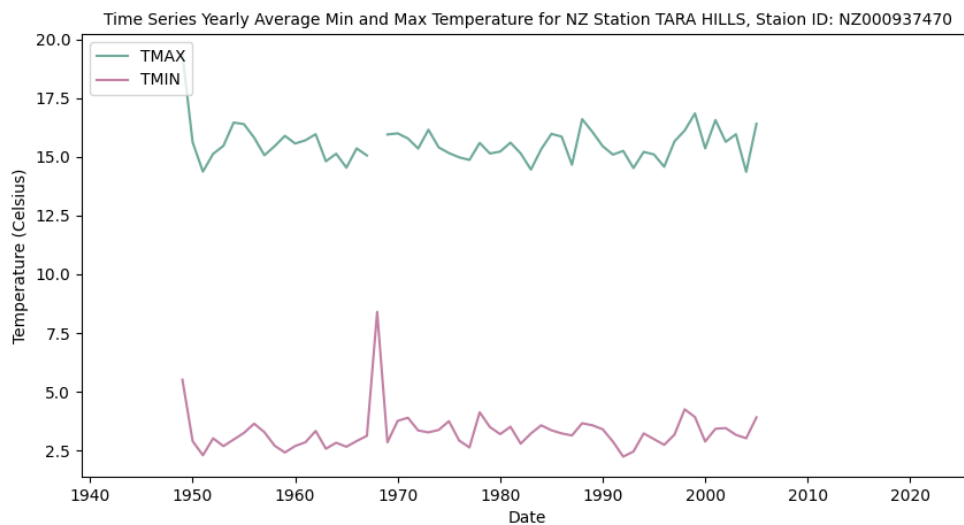
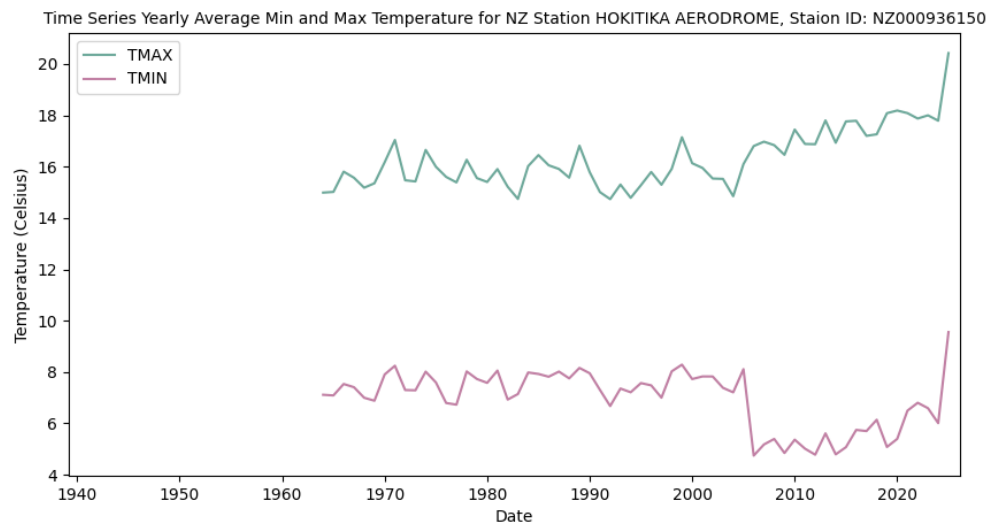
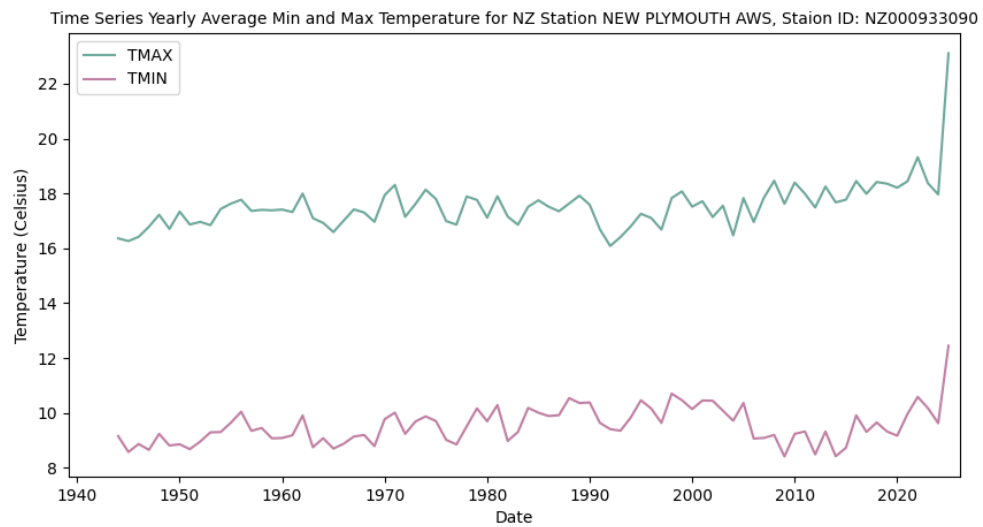


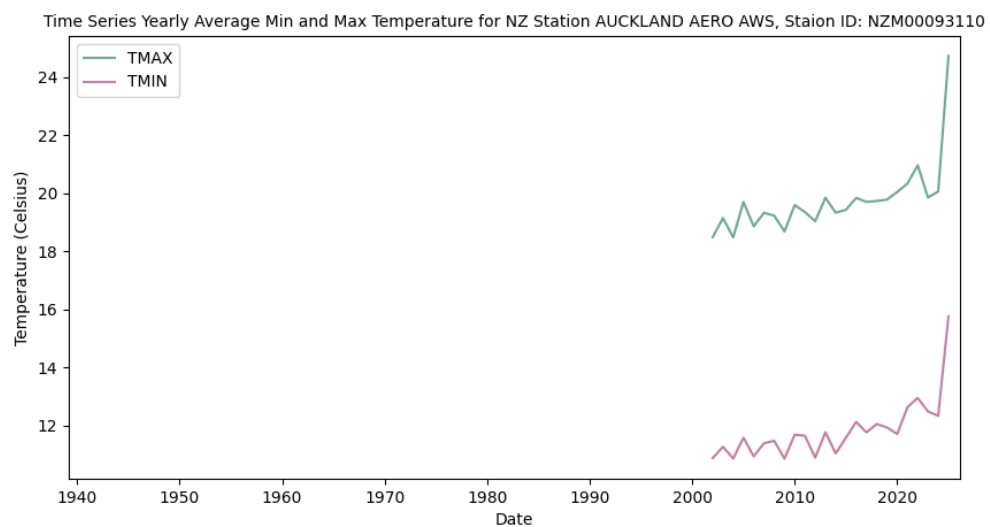
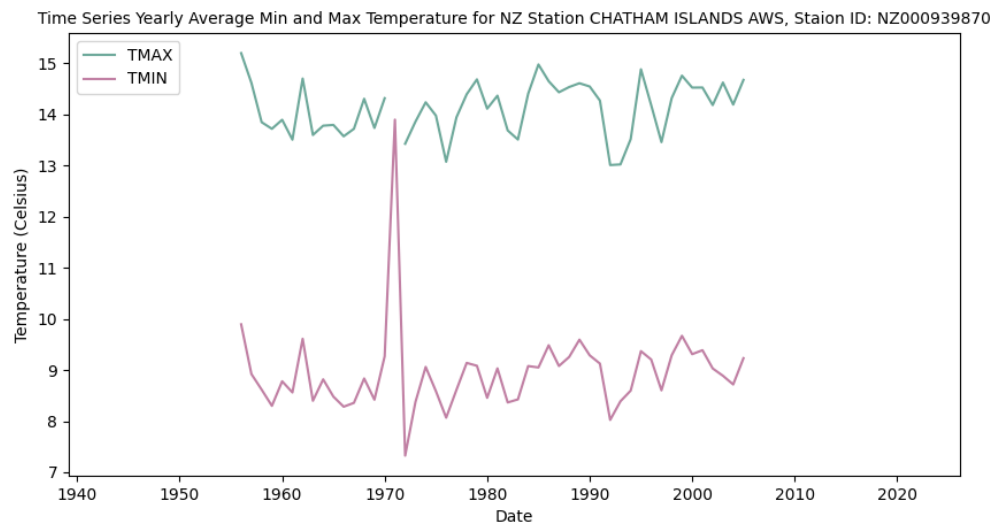
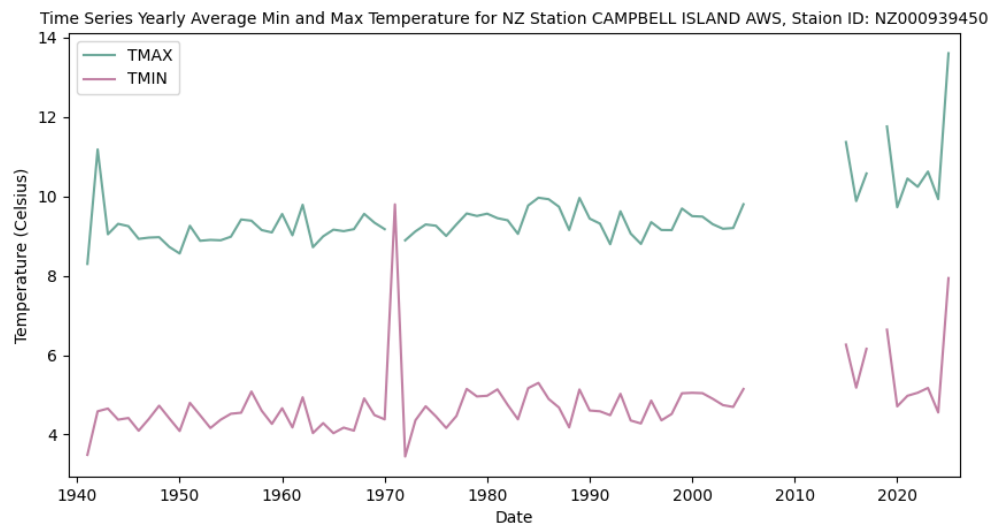


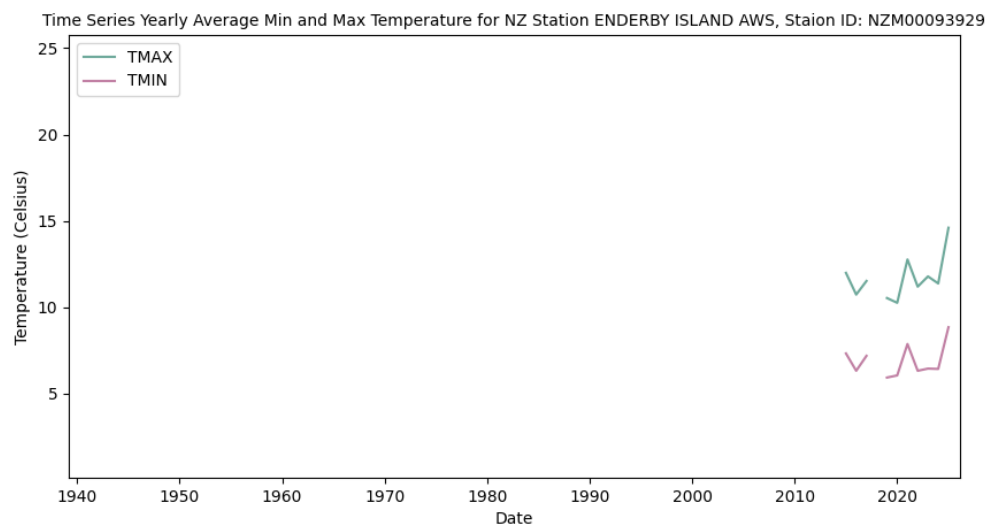
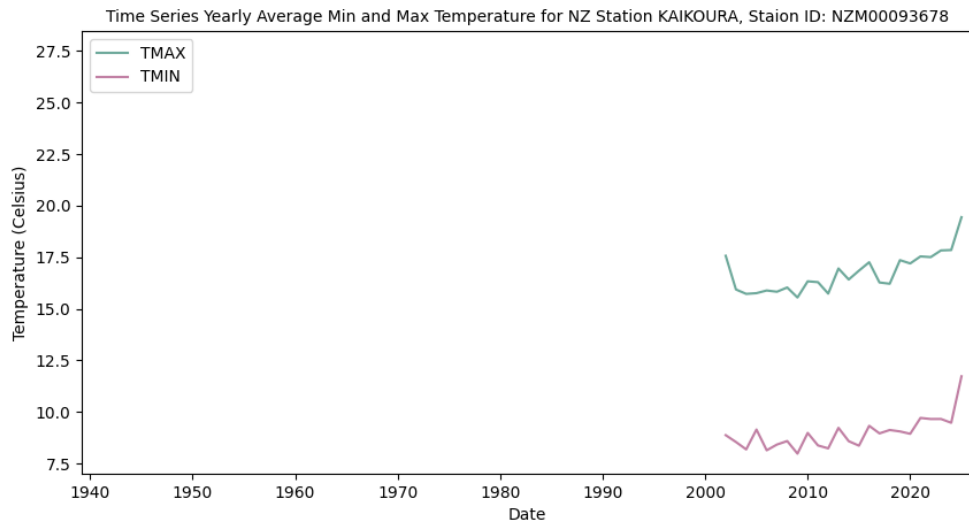
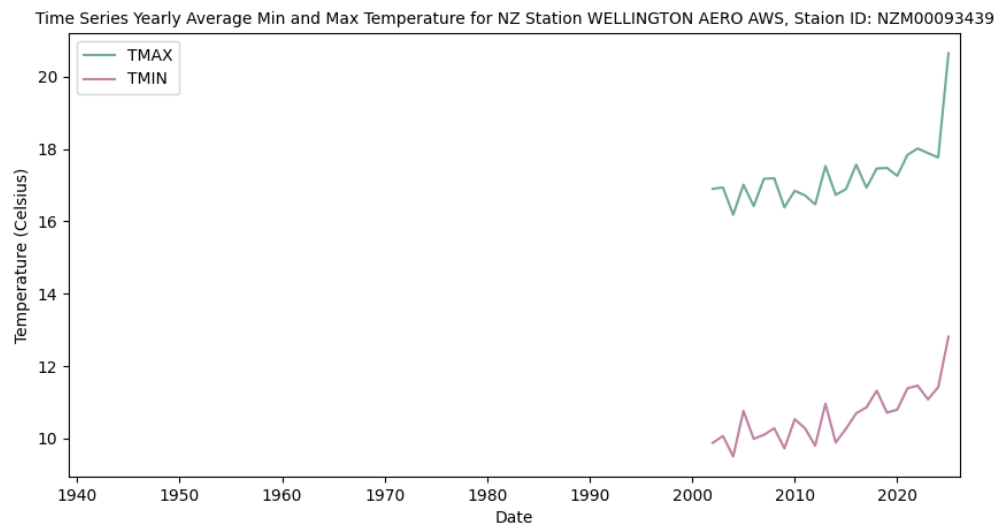


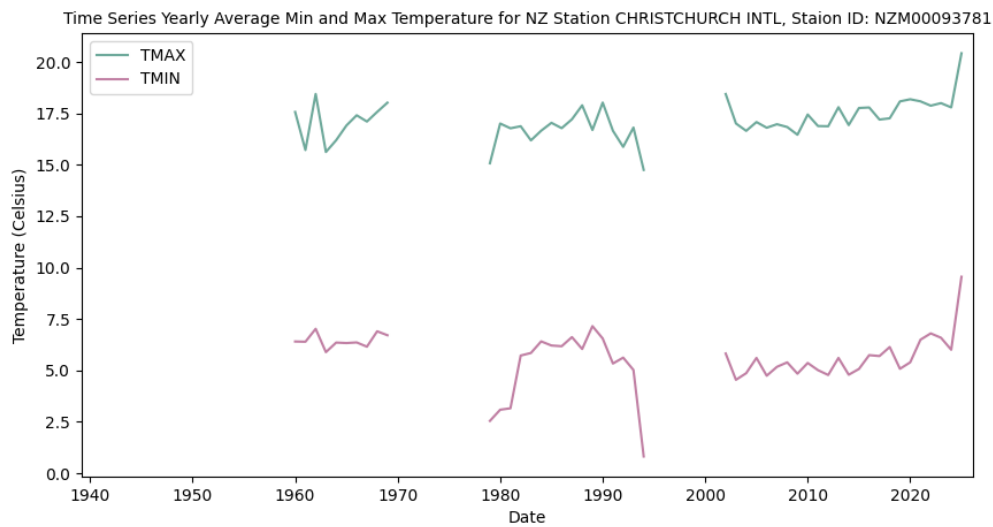












References

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope*, 68(2), 159.

Prabhakaran, S. (2023, August 5). How to detect outliers with z-score. Machine Learning Plus.

<https://www.machinelearningplus.com/machine-learning/how-to-detect-outliers-with-z-score/>